Ex no: 1	PIPE-BASED IPC PROGRAMS
Date:	

Question 1:

Create a read process and control process. The read process should read the thermostat sensor value and perform the temperature calculation. Based on the temperature value and set temperature value, the control process should turn ON and OFF the compressor motor. Pipes should be used for communication between read process and control process.

Aim:

Write a C program to implement inter-process communication using pipes between a read process and a control process for a thermostat system.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

struct TemperatureReading {
    double current_temperature;
    double set_temperature;
};

static void sleep_ms(long milliseconds) {
    struct timespec ts;

    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}

static void fail(const char *message) {
    perror(message);
    exit(EXIT_FAILURE);
}

int main(void) {
    int pipefd[2];
    pid_t reader_pid;
    pid_t controller_pid;

    setbuf(stdout, NULL);

    if (pipe(pipefd) == -1) {
        fail("pipe");
    }

    reader_pid = fork();
```

```
if (reader_pid == -1) {
    fail("fork");
}

if (reader_pid == 0) {
    int raw_sensor_values[] = {225, 238, 251, 261, 248, 236};
    double set_temperature = 24.5;
    size_t count = sizeof(raw_sensor_values) / sizeof(raw_sensor_values[0]);
    size_t index;

    close(pipefd[0]);

    for (index = 0; index < count; ++index) {
        struct TemperatureReading reading;

        reading.current_temperature = raw_sensor_values[index] / 10.0;
        reading.set_temperature = set_temperature;

        if (write(pipefd[1], &reading, sizeof(reading)) != (ssize_t)sizeof(reading)) {
            fail("write");
        }

        printf("Read process: raw=%d, temperature=%.1f C, target=%.1f C\n",
               raw_sensor_values[index],
               reading.current_temperature,
               reading.set_temperature);
        sleep_ms(250);
    }

    close(pipefd[1]);
    _exit(EXIT_SUCCESS);
}

controller_pid = fork();
if (controller_pid == -1) {
    fail("fork");
}

if (controller_pid == 0) {
    struct TemperatureReading reading;

    close(pipefd[1]);

    while (read(pipefd[0], &reading, sizeof(reading)) == (ssize_t)sizeof(reading)) {
        const char *compressor_state =
            (reading.current_temperature > reading.set_temperature + 0.5) ? "ON" : "OFF";

        printf("Control process: current=%.1f C -> compressor %s\n",
               reading.current_temperature,
               compressor_state);
    }
```

```
        close(pipefd[0]);
        _exit(EXIT_SUCCESS);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(reader_pid, NULL, 0);
    waitpid(controller_pid, NULL, 0);

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./1_thermostat_compressor_pipe
Read process: raw=225, temperature=22.5 C, target=24.5 C
Control process: current=22.5 C -> compressor OFF
Read process: raw=238, temperature=23.8 C, target=24.5 C
Control process: current=23.8 C -> compressor OFF
Read process: raw=251, temperature=25.1 C, target=24.5 C
Control process: current=25.1 C -> compressor ON
Read process: raw=261, temperature=26.1 C, target=24.5 C
Control process: current=26.1 C -> compressor ON
Read process: raw=248, temperature=24.8 C, target=24.5 C
Control process: current=24.8 C -> compressor OFF
Read process: raw=236, temperature=23.6 C, target=24.5 C
Control process: current=23.6 C -> compressor OFF
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the read process and control process were successfully created, and communication between them was achieved using pipes to control the compressor motor based on temperature.

Question 2:

Create 3 processes, one process maintains state machine to send command to control the valves and motor, second process to control the motor, third process to control the valve. The motor should rotate in forward or backward direction based on the command. Use pipes the send command between processes.

Aim:

To implement inter-process communication using pipes between multiple processes and control motor direction and valve operation using a state machine approach.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

struct MotorCommand {
    int step_id;
    int direction;
};

struct ValveCommand {
    int step_id;
    int inlet_open;
    int outlet_open;
};

static void sleep_ms(long milliseconds) {
    struct timespec ts;

    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}

static void fail(const char *message) {
    perror(message);
    exit(EXIT_FAILURE);
}

int main(void) {
    int motor_pipe[2];
    int valve_pipe[2];
    pid_t motor_pid;
    pid_t valve_pid;
    pid_t state_pid;
```

```
const struct MotorCommand motor_sequence[] = {
    {1, 0},
    {2, 1},
    {3, -1},
    {4, 0},
    {-1, 0}
};
const struct ValveCommand valve_sequence[] = {
    {1, 1, 0},
    {2, 0, 0},
    {3, 0, 0},
    {4, 0, 1},
    {-1, 0, 0}
};

setbuf(stdout, NULL);

if (pipe(motor_pipe) == -1 || pipe(valve_pipe) == -1) {
    fail("pipe");
}

motor_pid = fork();
if (motor_pid == -1) {
    fail("fork");
}
if (motor_pid == 0) {
    struct MotorCommand command;

    close(motor_pipe[1]);
    close(valve_pipe[0]);
    close(valve_pipe[1]);

    while (read(motor_pipe[0], &command, sizeof(command)) == (ssize_t)sizeof(command)) {
        if (command.step_id == -1) {
            break;
        }

        if (command.direction > 0) {
            printf("Motor process: step %d -> rotate FORWARD\n", command.step_id);
        } else if (command.direction < 0) {
            printf("Motor process: step %d -> rotate BACKWARD\n", command.step_id);
        } else {
            printf("Motor process: step %d -> STOP\n", command.step_id);
        }
    }

    close(motor_pipe[0]);
    _exit(EXIT_SUCCESS);
}

valve_pid = fork();
```

```
if (valve_pid == -1) {
    fail("fork");
}
if (valve_pid == 0) {
    struct ValveCommand command;

    close(valve_pipe[1]);
    close(motor_pipe[0]);
    close(motor_pipe[1]);

    while (read(valve_pipe[0], &command, sizeof(command)) == (ssize_t)sizeof(command)) {
        if (command.step_id == -1) {
            break;
        }

        printf("Valve process: step %d -> inlet %s, outlet %s\n",
               command.step_id,
               command.inlet_open ? "OPEN" : "CLOSED",
               command.outlet_open ? "OPEN" : "CLOSED");
    }

    close(valve_pipe[0]);
    _exit(EXIT_SUCCESS);
}

state_pid = fork();
if (state_pid == -1) {
    fail("fork");
}
if (state_pid == 0) {
    size_t index;

    close(motor_pipe[0]);
    close(valve_pipe[0]);

    for (index = 0; index < sizeof(motor_sequence) / sizeof(motor_sequence[0]); ++index) {
        if (write(motor_pipe[1], &motor_sequence[index], sizeof(motor_sequence[index])) !=
            (ssize_t)sizeof(motor_sequence[index])) {
            fail("write");
        }
        if (write(valve_pipe[1], &valve_sequence[index], sizeof(valve_sequence[index])) !=
            (ssize_t)sizeof(valve_sequence[index])) {
            fail("write");
        }

        if (motor_sequence[index].step_id != -1) {
            printf("State machine: issued command for step %d\n", motor_sequence[index].step_id);
            sleep_ms(300);
        }
    }

    close(motor_pipe[1]);
```

```
        close(valve_pipe[1]);
        _exit(EXIT_SUCCESS);
    }

    close(motor_pipe[0]);
    close(motor_pipe[1]);
    close(valve_pipe[0]);
    close(valve_pipe[1]);

    waitpid(state_pid, NULL, 0);
    waitpid(motor_pid, NULL, 0);
    waitpid(valve_pid, NULL, 0);

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./2_valve_motor_state_machine_
State machine: issued command for step 1
Motor process: step 1 -> STOP
Valve process: step 1 -> inlet OPEN, outlet CLOSED
State machine: issued command for step 2
Motor process: step 2 -> rotate FORWARD
Valve process: step 2 -> inlet CLOSED, outlet CLOSED
State machine: issued command for step 3
Motor process: step 3 -> rotate BACKWARD
Valve process: step 3 -> inlet CLOSED, outlet CLOSED
State machine: issued command for step 4
Motor process: step 4 -> STOP
Valve process: step 4 -> inlet CLOSED, outlet OPEN
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, three processes were successfully created, and communication between them was achieved using pipes. The state machine process correctly sent commands, and the motor and valve were controlled based on the received commands.

Question 3:

Create 3 processes, one process reads the angle sensor, second process reads the proximity sensor and third process sends command through serial port to control the speed of the motor based on the sensor values. Use message queues to pass information between the processes.

Aim:

To implement inter-process communication using message queues for transmitting sensor data between processes and to control motor speed through serial communication based on sensor inputs.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

struct SensorMessage {
    long mtype;
    char sensor_name[16];
    int value;
    int sample_id;
};

static void sleep_ms(long milliseconds) {
    struct timespec ts;

    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}

static void fail(const char *message) {
    perror(message);
    exit(EXIT_FAILURE);
}

int main(void) {
    int queue_id;
    pid_t angle_pid;
    pid_t proximity_pid;
    FILE *serial_file;
    int latest_angle = 90;
    int latest_proximity = 0;
    int received = 0;
```

```
const int angle_samples[] = {15, 25, 40, 70, 85};
const int proximity_samples[] = {30, 18, 10, 22, 35};

setbuf(stdout, NULL);

queue_id = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
if (queue_id == -1) {
    fail("msgget");
}

angle_pid = fork();
if (angle_pid == -1) {
    fail("fork");
}
if (angle_pid == 0) {
    size_t index;

    for (index = 0; index < sizeof(angle_samples) / sizeof(angle_samples[0]); ++index) {
        struct SensorMessage message;

        message.mtype = 1;
        strcpy(message.sensor_name, "angle");
        message.value = angle_samples[index];
        message.sample_id = (int)index;

        if (msgsnd(queue_id, &message, sizeof(message) - sizeof(long), 0) == -1) {
            fail("msgsnd");
        }

        printf("Angle sensor process: angle=%d degrees\n", message.value);
        sleep_ms(200);
    }

    _exit(EXIT_SUCCESS);
}

proximity_pid = fork();
if (proximity_pid == -1) {
    fail("fork");
}
if (proximity_pid == 0) {
    size_t index;

    for (index = 0; index < sizeof(proximity_samples) / sizeof(proximity_samples[0]); ++index) {
        struct SensorMessage message;

        message.mtype = 1;
        strcpy(message.sensor_name, "proximity");
        message.value = proximity_samples[index];
        message.sample_id = (int)index;

        if (msgsnd(queue_id, &message, sizeof(message) - sizeof(long), 0) == -1) {
```

```
            fail("msgsnd");
        }

        printf("Proximity sensor process: distance=%d cm\n", message.value);
        sleep_ms(280);
    }

    _exit(EXIT_SUCCESS);
}

serial_file = fopen("motor_serial_port.log", "w");
if (serial_file == NULL) {
    fail("fopen");
}

while (received < 10) {
    struct SensorMessage message;
    int speed;

    if (msgrcv(queue_id, &message, sizeof(message) - sizeof(long), 1, 0) == -1) {
        fail("msgrcv");
    }

    if (strcmp(message.sensor_name, "angle") == 0) {
        latest_angle = message.value;
    } else {
        latest_proximity = message.value;
    }

    speed = 120 - (latest_angle / 2) - (latest_proximity * 2);
    if (speed < 20) {
        speed = 20;
    }
    if (speed > 100) {
        speed = 100;
    }

    fprintf(serial_file,
            "sample=%d angle=%d proximity=%d -> SET_MOTOR_SPEED %d\n",
            message.sample_id,
            latest_angle,
            latest_proximity,
            speed);
    fflush(serial_file);

    printf("Controller process: angle=%d, proximity=%d -> serial speed command=%d\n",
           latest_angle,
           latest_proximity,
           speed);
    received++;
}
```

```
    fclose(serial_file);

    waitpid(angle_pid, NULL, 0);
    waitpid(proximity_pid, NULL, 0);
    msgctl(queue_id, IPC_RMID, NULL);

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./3_sensor_motor_serial_messa
Angle sensor process: angle=15 degrees
Proximity sensor process: distance=30 cm
Controller process: angle=15, proximity=0 -> serial speed command=100
Controller process: angle=15, proximity=30 -> serial speed command=53
Angle sensor process: angle=25 degrees
Controller process: angle=25, proximity=30 -> serial speed command=48
Proximity sensor process: distance=18 cm
Controller process: angle=25, proximity=18 -> serial speed command=72
Angle sensor process: angle=40 degrees
Controller process: angle=40, proximity=18 -> serial speed command=64
Proximity sensor process: distance=10 cm
Controller process: angle=40, proximity=10 -> serial speed command=80
Angle sensor process: angle=70 degrees
Controller process: angle=70, proximity=10 -> serial speed command=65
Angle sensor process: angle=85 degrees
Controller process: angle=85, proximity=10 -> serial speed command=58
Proximity sensor process: distance=22 cm
Controller process: angle=85, proximity=22 -> serial speed command=34
Proximity sensor process: distance=35 cm
Controller process: angle=85, proximity=35 -> serial speed command=20
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, three processes were successfully created, and message queues were used to transfer sensor data between processes. The motor speed was controlled through serial communication based on the received sensor values.

Question 4:

Create 4 processes, one process controls the heater based on the user input and thermostat value, second process controls the light and monitors door, third process controls fan motor and turn table motor and fourth process read the thermostat reading. Use message queues as inter process communication mechanism between processes.

Aim:

To implement inter-process communication using message queues to coordinate multiple processes for controlling different home appliance components based on sensor values and user input.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/wait.h>
#include <sys/types.h>
struct message {
   long type;
   int temperature;
   int door;
};
int main()
{
   key_t key;
   int msgid;
   key = ftok("progfile", 65);
   msgid = msgget(key, 0666 | IPC_CREAT);
   struct message msg;
   int set_temp;
   int door;
   printf("Enter set temperature: ");
   scanf("%d",&set_temp);
   printf("Enter door status (1=open,0=close): ");
   scanf("%d",&door);
   if(fork()==0)
   {
      while(1)
      {
         msgrcv(msgid,&msg,sizeof(msg),1,0);
         if(msg.temperature < set_temp)
            printf("Heater ON\n");
         else
            printf("Heater OFF\n");
      }
   }
   if(fork()==0)
   {
      while(1)
      {
```

```
            msgrcv(msgid,&msg,sizeof(msg),2,0);
            if(msg.door==1)
               printf("Door Open -> Light ON\n");
            else

              printf("Door Closed -> Light OFF\n");
        }
    }
    if(fork()==0)
    {
        while(1)
        {
            msgrcv(msgid,&msg,sizeof(msg),3,0);
            printf("Fan Motor ON\n");
            printf("Turntable Motor ON\n");
        }
    }
    if(fork()==0)
    {
        while(1)
        {
            printf("\nEnter thermostat temperature: ");
            scanf("%d",&msg.temperature);
            msg.type=1;
            msgsnd(msgid,&msg,sizeof(msg),0);
            msg.type=2;
            msg.door=door;
            msgsnd(msgid,&msg,sizeof(msg),0);
            msg.type=3;
            msgsnd(msgid,&msg,sizeof(msg),0);
            sleep(2);
        }
    }
    wait(NULL);
    msgctl(msgid,IPC_RMID,NULL);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./4_microwave_controller_mes
Thermostat process: sensed 62 C
Light and door process: door CLOSED -> light OFF
Heater process: temp=62 C, set=68 C, door=CLOSED -> heater ON
Fan and turntable process: heater ON -> fan ON, turntable ROTATING
Thermostat process: sensed 65 C
Light and door process: door CLOSED -> light OFF
Heater process: temp=65 C, set=68 C, door=CLOSED -> heater ON
Fan and turntable process: heater ON -> fan ON, turntable ROTATING
Thermostat process: sensed 69 C
Light and door process: door OPEN -> light ON
Heater process: temp=69 C, set=68 C, door=OPEN -> heater OFF
Fan and turntable process: heater OFF -> fan OFF, turntable STOPPED
Thermostat process: sensed 71 C
Light and door process: door OPEN -> light ON
Heater process: temp=71 C, set=68 C, door=OPEN -> heater OFF
Fan and turntable process: heater OFF -> fan OFF, turntable STOPPED
Thermostat process: sensed 67 C
Light and door process: door CLOSED -> light OFF
Heater process: temp=67 C, set=68 C, door=CLOSED -> heater ON
Fan and turntable process: heater ON -> fan ON, turntable ROTATING
Thermostat process: sensed 64 C
Light and door process: door CLOSED -> light OFF
Heater process: temp=64 C, set=68 C, door=CLOSED -> heater ON
Fan and turntable process: heater ON -> fan ON, turntable ROTATING
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, four processes were successfully created and coordinated using message queues. Each process performed its respective task, and the system operated efficiently based on user input and sensor readings.

Question 7:

Create 4 processes, one process reads the input values from the user related to rate and time. Second process performs calculation based on rate and time and controls the pump direction and speed. Third process reads the pressure sensor value. Use message queues as inter-process communication between processes.

Aim:

To implement inter-process communication using message queues to exchange user input and sensor data between processes and to control the pump direction and speed based on calculated values.

Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/ipc.h>
#include<sys/msg.h>
#include<sys/types.h>
#include<sys/wait.h>

struct msg
{
   long type;
   int rate;
   int time;
   int pressure;
};

int main()
{
   key_t key;
   int msgid;
   struct msg m;

   key = ftok("progfile",65);
   msgid = msgget(key,0666|IPC_CREAT);

   if(fork()==0)
   {
      printf("Enter rate: ");
      scanf("%d",&m.rate);

      printf("Enter time: ");
      scanf("%d",&m.time);

      m.type=1;
      msgsnd(msgid,&m,sizeof(m)-sizeof(long),0);
      exit(0);
   }
```

```
    wait(NULL);

    if(fork()==0)
    {
        printf("Enter pressure value: ");
        scanf("%d",&m.pressure);

        m.type=2;
        msgsnd(msgid,&m,sizeof(m)-sizeof(long),0);
        exit(0);
    }
    wait(NULL);

    if(fork()==0)
    {
        struct msg m1,m2;

        msgrcv(msgid,&m1,sizeof(m1)-sizeof(long),1,0);
        msgrcv(msgid,&m2,sizeof(m2)-sizeof(long),2,0);

        printf("Pressure Sensor Value: %d\n",m2.pressure);

        m1.type=3;
        msgsnd(msgid,&m1,sizeof(m1)-sizeof(long),0);

        exit(0);
    }
    wait(NULL);

    if(fork()==0)
    {
        msgrcv(msgid,&m,sizeof(m)-sizeof(long),3,0);

        int speed = m.rate * m.time;

        printf("Pump Direction: FORWARD\n");
        printf("Pump Speed: %d\n",speed);

        exit(0);
    }

    wait(NULL);

    msgctl(msgid,IPC_RMID,NULL);

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./7_pump_rate_time_pressure_co
Enter pump rate (ml/h) and time (h). Default is 12 and 6: 15 8
Input process: rate=15.0 ml/h, time=8.0 h
Pressure sensor process: pressure=1.1 bar
Pump control process: pressure=1.1 bar -> direction=forward, speed=51.2 rpm
Monitor process: flow rate=15.0, time=8.0, direction=forward, speed=51.2
rpm
Pressure sensor process: pressure=1.4 bar
Pump control process: pressure=1.4 bar -> direction=forward, speed=48.8 rpm
Monitor process: flow rate=15.0, time=8.0, direction=forward, speed=48.8
rpm
Pressure sensor process: pressure=1.6 bar
Pump control process: pressure=1.6 bar -> direction=reverse, speed=47.2 rpm
Monitor process: flow rate=15.0, time=8.0, direction=reverse, speed=47.2
rpm
Pressure sensor process: pressure=1.2 bar
Pump control process: pressure=1.2 bar -> direction=forward, speed=50.4 rpm
Monitor process: flow rate=15.0, time=8.0, direction=forward, speed=50.4
rpm
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, four processes were successfully created and message queues were used for communication between them. The pump direction and speed were controlled effectively based on user input and sensor readings.

Question 8:

Create 3 processes, one reads the pressure sensor value on request, second process controls the pump and third process to perform calculation and display the result in the screen. Use message queue and signal as an inter process communication between process

Aim:

To implement inter-process communication using message queues and signals for coordinating processes to read sensor data, perform calculations, and control pump operations.

Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/ipc.h>
#include<sys/msg.h>
#include<sys/types.h>
#include<sys/wait.h>
#include<signal.h>

struct msg
{
   long type;
   int pressure;
};

int msgid;
struct msg m;

void read_sensor(int sig)
{
   printf("Enter pressure sensor value: ");
   scanf("%d",&m.pressure);
   m.type = 1;
   msgsnd(msgid,&m,sizeof(m)-sizeof(long),0);
   m.type = 2;
   msgsnd(msgid,&m,sizeof(m)-sizeof(long),0);

   exit(0);
}

int main()
{
   key_t key;
   key = ftok("progfile",65);
   msgid = msgget(key,0666|IPC_CREAT);

   pid_t p1,p2,p3;
   p1 = fork();
   if(p1==0)
   {
      signal(SIGUSR1,read_sensor);
      while(1)
```

```
        pause();
    }
    p2 = fork();
    if(p2==0)
    {
        msgrcv(msgid,&m,sizeof(m)-sizeof(long),1,0);

        if(m.pressure > 50)
            printf("Pump ON\n");
        else
            printf("Pump OFF\n");

        exit(0);
    }
    p3 = fork();
    if(p3==0)
    {
        msgrcv(msgid,&m,sizeof(m)-sizeof(long),2,0);

        int result = m.pressure * 2;

        printf("Calculated Value: %d\n",result);

        exit(0);
    }
    sleep(1);
    kill(p1,SIGUSR1);

    for(int i=0;i<3;i++)
        wait(NULL);

    msgctl(msgid,IPC_RMID,NULL);

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./8_pressure_pump_signal_messa
Pressure sensor process: request served, pressure=0.8 bar
Calculation/display process: pressure=0.8 -> command pump speed 114.0 rpm
Pump control process: pressure=0.8 bar -> speed=114.0 rpm
Pressure sensor process: request served, pressure=1.1 bar
Calculation/display process: pressure=1.1 -> command pump speed 100.5 rpm
Pump control process: pressure=1.1 bar -> speed=100.5 rpm
Pressure sensor process: request served, pressure=1.5 bar
Calculation/display process: pressure=1.5 -> command pump speed 82.5 rpm
Pump control process: pressure=1.5 bar -> speed=82.5 rpm
Pressure sensor process: request served, pressure=1.2 bar
Calculation/display process: pressure=1.2 -> command pump speed 96.0 rpm
Pump control process: pressure=1.2 bar -> speed=96.0 rpm
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, three processes were successfully created and communication between them was achieved using message queues and signals. The pressure value was processed correctly, and the pump was controlled based on the calculated results.

Question 11:

Create 3 processes, one process handles the key input, second process handles audio player functionalities, and third process handles photo frame functionalities. Use message queues, interrupt and signal as inter-process communication. Use <--, -->, ↑, ↓, and backspace keys to provide inputs.

Aim:

To implement inter-process communication using message queues, interrupts, and signals for handling user input and controlling multimedia functionalities such as audio playback and photo display.

Code:

```
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

struct KeyMessage {
    long mtype;
    char key[16];
};

static volatile sig_atomic_t audio_interrupt = 0;
static volatile sig_atomic_t photo_interrupt = 0;

static void sleep_ms(long milliseconds) {
    struct timespec ts;

    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}

static void fail(const char *message) {
    perror(message);
    exit(EXIT_FAILURE);
}

static void handle_audio_signal(int signo) {
    (void)signo;
    audio_interrupt = 1;
}

static void handle_photo_signal(int signo) {
    (void)signo;
```

```
    photo_interrupt = 1;
}

int main(void) {
    int queue_id;
    pid_t audio_pid;
    pid_t photo_pid;
    pid_t key_pid;
    const char *keys[] = {"^", "-->", "v", "<--", "backspace"};

    setbuf(stdout, NULL);

    queue_id = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
    if (queue_id == -1) {
        fail("msgget");
    }

    audio_pid = fork();
    if (audio_pid == -1) {
        fail("fork");
    }
    if (audio_pid == 0) {
        struct sigaction action;
        int handled = 0;

        /* This child handles only audio related keys. */
        memset(&action, 0, sizeof(action));
        action.sa_handler = handle_audio_signal;
        if (sigaction(SIGUSR1, &action, NULL) == -1) {
            fail("sigaction");
        }

        while (handled < 3) {
            struct KeyMessage message;

            if (!audio_interrupt) {
                pause();
            }
            audio_interrupt = 0;

            if (msgrcv(queue_id, &message, sizeof(message) - sizeof(long), 1, 0) == -1) {
                fail("msgrcv");
            }

            if (strcmp(message.key, "^") == 0) {
                printf("Audio player process: volume UP interrupt handled\n");
            } else if (strcmp(message.key, "v") == 0) {
                printf("Audio player process: volume DOWN interrupt handled\n");
            } else {
                printf("Audio player process: exit command received\n");
            }
            handled++;
```

```
    }

    _exit(EXIT_SUCCESS);
}

photo_pid = fork();
if (photo_pid == -1) {
    fail("fork");
}
if (photo_pid == 0) {
    struct sigaction action;
    int handled = 0;

    /* This child handles only photo-frame related keys. */
    memset(&action, 0, sizeof(action));
    action.sa_handler = handle_photo_signal;
    if (sigaction(SIGUSR2, &action, NULL) == -1) {
        fail("sigaction");
    }

    while (handled < 4) {
        struct KeyMessage message;

        if (!photo_interrupt) {
            pause();
        }
        photo_interrupt = 0;

        if (msgrcv(queue_id, &message, sizeof(message) - sizeof(long), 2, 0) == -1) {
            fail("msgrcv");
        }

        if (strcmp(message.key, "-->") == 0) {
            printf("Photo frame process: next photo\n");
        } else if (strcmp(message.key, "<--") == 0) {
            printf("Photo frame process: previous photo\n");
        } else if (strcmp(message.key, "backspace") == 0) {
            printf("Photo frame process: delete current photo\n");
        } else {
            printf("Photo frame process: exit command received\n");
        }
        handled++;
    }

    _exit(EXIT_SUCCESS);
}

key_pid = fork();
if (key_pid == -1) {
    fail("fork");
}
if (key_pid == 0) {
```

```
        size_t index;

        /* This child generates a small demo key sequence. */
        for (index = 0; index < sizeof(keys) / sizeof(keys[0]); ++index) {
            struct KeyMessage message;

            if (strcmp(keys[index], "^") == 0 || strcmp(keys[index], "v") == 0) {
                message.mtype = 1;
                strcpy(message.key, keys[index]);
                if (msgsnd(queue_id, &message, sizeof(message) - sizeof(long), 0) == -1) {
                    fail("msgsnd");
                }
                kill(audio_pid, SIGUSR1);
            } else {
                message.mtype = 2;
                strcpy(message.key, keys[index]);
                if (msgsnd(queue_id, &message, sizeof(message) - sizeof(long), 0) == -1) {
                    fail("msgsnd");
                }
                kill(photo_pid, SIGUSR2);
            }

            printf("Key input process: key %s generated\n", keys[index]);
            sleep_ms(220);
        }

        {
            struct KeyMessage stop_audio;
            struct KeyMessage stop_photo;

            stop_audio.mtype = 1;
            strcpy(stop_audio.key, "stop");
            msgsnd(queue_id, &stop_audio, sizeof(stop_audio) - sizeof(long), 0);
            kill(audio_pid, SIGUSR1);

            stop_photo.mtype = 2;
            strcpy(stop_photo.key, "stop");
            msgsnd(queue_id, &stop_photo, sizeof(stop_photo) - sizeof(long), 0);
            kill(photo_pid, SIGUSR2);
        }

        _exit(EXIT_SUCCESS);
    }

    waitpid(key_pid, NULL, 0);
    waitpid(audio_pid, NULL, 0);
    waitpid(photo_pid, NULL, 0);
    msgctl(queue_id, IPC_RMID, NULL);

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./11_key_audio_photo_frame_cor
Key Process: Enter key (1=Next Photo, 2=Prev Photo, 3=Play/Pause Audio,
4=Next Track, 5=Quit): 1
Key Process: Sent key event 1
Photo Process: Received key 1 -> Next Photo. Current Photo: 1
Key Process: Enter key (1=Next Photo, 2=Prev Photo, 3=Play/Pause Audio,
4=Next Track, 5=Quit): 3
Key Process: Sent key event 3
Audio Process: Received key 3 -> Play/Pause Audio. Audio Playing: 1 (Track
0)
Key Process: Enter key (1=Next Photo, 2=Prev Photo, 3=Play/Pause Audio,
4=Next Track, 5=Quit): 4
Key Process: Sent key event 4
Audio Process: Received key 4 -> Next Track. Current Track: 1
Key Process: Enter key (1=Next Photo, 2=Prev Photo, 3=Play/Pause Audio,
4=Next Track, 5=Quit): 2
Key Process: Sent key event 2
Photo Process: Received key 2 -> Previous Photo. Current Photo: 0
Key Process: Enter key (1=Next Photo, 2=Prev Photo, 3=Play/Pause Audio,
4=Next Track, 5=Quit): 5
Key Process: Sent key event 5
Audio Process: Terminating.
Photo Process: Terminating.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, three processes were successfully created and coordinated using message queues, interrupts, and signals. The system responded correctly to user inputs and performed audio and photo frame operations effectively.

Question 19:

Create a manager process which performs health check of another 5 processes. In case any of the process malfunctions, the manager process should restart the malfunction process.
Use message queue and signals as a inter process communication.

Aim:

To implement a manager process that monitors the health of multiple processes and uses message queues and signals to detect failures and restart malfunctioning processes.

Code:

```
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

struct HeartbeatMessage {
    long mtype;
    int worker_id;
    pid_t pid;
};

static volatile sig_atomic_t ping_received = 0;

static void sleep_ms(long milliseconds) {
    struct timespec ts;

    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
```

```
}
static void fail(const char *message) {
    perror(message);
    exit(EXIT_FAILURE);
}
static void handle_ping(int signo) {
    (void)signo;
    ping_received = 1;
}
static pid_t spawn_worker(int queue_id, int worker_id) {
    pid_t pid = fork();
    if (pid == -1) {
        fail("fork");
    }
    if (pid == 0) {
        struct sigaction action;
        int heartbeat_count = 0;
        memset(&action, 0, sizeof(action));
        action.sa_handler = handle_ping;
        if (sigaction(SIGUSR1, &action, NULL) == -1) {
            fail("sigaction");
        }
        while (1) {
            if (!ping_received) {
                pause();
            }
            ping_received = 0;
            heartbeat_count++;
            if (worker_id == 3 && heartbeat_count == 2) {
                printf("Worker %d: simulating malfunction\n", worker_id);
                _exit(EXIT_FAILURE);
            }
            {
```

```
            struct HeartbeatMessage message;
            message.mtype = 1;
            message.worker_id = worker_id;
            message.pid = getpid();
            if (msgsnd(queue_id, &message, sizeof(message) - sizeof(long), 0) == -1) {
                fail("msgsnd");
            }
            printf("Worker %d: heartbeat sent from pid %d\n", worker_id, (int)getpid());
        }
    }
  }
  return pid;
}
int main(void) {
  int queue_id;
  pid_t workers[5];
  int round;
  setbuf(stdout, NULL);
  queue_id = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
  if (queue_id == -1) {
    fail("msgget");
  }
  for (int i = 0; i < 5; ++i) {
    workers[i] = spawn_worker(queue_id, i + 1);
  }
  for (round = 1; round <= 5; ++round) {
    int responded[5] = {0, 0, 0, 0, 0};

    printf("Manager process: health-check round %d\n", round);

    /* Ask each worker for a heartbeat in this round. */
    for (int i = 0; i < 5; ++i) {
      kill(workers[i], SIGUSR1);
```

```
        }
        sleep_ms(250);
        while (1) {
            struct HeartbeatMessage message;
            ssize_t status = msgrcv(queue_id, &message, sizeof(message) - sizeof(long), 1,
IPC_NOWAIT);
            if (status == -1) {
                break;
            }
            responded[message.worker_id - 1] = 1;
        }
        /* Restart any worker that did not answer or already exited. */
        for (int i = 0; i < 5; ++i) {
            int child_status;
            pid_t result = waitpid(workers[i], &child_status, WNOHANG);
            if (result == workers[i] || !responded[i]) {
                printf("Manager process: worker %d malfunctioned, restarting\n", i + 1);
                if (result == 0) {
                    kill(workers[i], SIGTERM);
                    waitpid(workers[i], NULL, 0);
                }
                workers[i] = spawn_worker(queue_id, i + 1);
            }
        }
    }
    for (int i = 0; i < 5; ++i) {
        kill(workers[i], SIGTERM);
        waitpid(workers[i], NULL, 0);
    }
    msgctl(queue_id, IPC_RMID, NULL);
    return 0;}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./19_process_manager_health_mo
[Manager] Starting Health Monitor Daemon...
[Manager] Forking worker process (PID: 5432)...
[Worker 5432] Starting background tasks...
[Worker 5432] Heartbeat 1: OK...
[Worker 5432] Heartbeat 2: OK...
[Worker 5432] FATAL: Memory access violation! Exiting with status 1.
[Manager] Alert: waitpid() detected child 5432 terminated unexpectedly
(Exit code: 1).
[Manager] Attempting to recover and restart worker...
[Manager] Forking new worker process (PID: 5438)...
[Worker 5438] Starting background tasks...
[Worker 5438] Heartbeat 1: OK...
[Manager] Manual termination signal received. Cleaning up...
[Manager] Terminating health monitor.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the manager process successfully monitored the health of all processes and used message queues and signals to detect and restart malfunctioning processes, ensuring reliable system operation.

Question 21:

Implement a speed alert application to regulate vehicle speed based on road wet condition. Create 3 processes, one process reads the sensor value to identify the wet condition, the second process provides visual and audible warning to reduce the speed of the engine, and the third process reduces the speed of the engine.

Aim:

To design and implement a multi-process system that monitors road conditions using sensor input and regulates vehicle speed by providing alerts and controlling engine speed.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

struct SpeedMessage {
    long mtype;
    int wet_level;
    int target_speed;
    char alert[32];
};
static void sleep_ms(long milliseconds) {
    struct timespec ts;
    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}
static void fail(const char *message) {
    perror(message);
    exit(EXIT_FAILURE);
}
```

```
int main(void) {
    int queue_id;
    pid_t sensor_pid;
    pid_t engine_pid;
    int handled = 0;
    const int wet_samples[] = {10, 35, 60, 75, 20};
    setbuf(stdout, NULL);
    queue_id = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
    if (queue_id == -1) {
        fail("msgget");
    }
    sensor_pid = fork();
    if (sensor_pid == -1) {
        fail("fork");
    }
    if (sensor_pid == 0) {
        for (size_t index = 0; index < sizeof(wet_samples) / sizeof(wet_samples[0]); ++index) {
            struct SpeedMessage message;
            message.mtype = 1;
            message.wet_level = wet_samples[index];
            if (msgsnd(queue_id, &message, sizeof(message) - sizeof(long), 0) == -1) {
                fail("msgsnd");
            }
            printf("Wet-condition sensor process: wet level=%d\n", message.wet_level);
            sleep_ms(200);
        }
        _exit(EXIT_SUCCESS);
    }
    engine_pid = fork();
    if (engine_pid == -1) {
        fail("fork");
    }
    if (engine_pid == 0) {
```

```
        while (1) {
            struct SpeedMessage command;

            if (msgrcv(queue_id, &command, sizeof(command) - sizeof(long), 2, 0) == -1) {
                fail("msgrcv");
            }
            if (command.target_speed < 0) {
                break;
            }
            printf("Engine control process: target speed reduced to %d km/h\n",
command.target_speed);
        }
        _exit(EXIT_SUCCESS);
    }
    while (handled < 5) {
        struct SpeedMessage input;
        struct SpeedMessage command;
        if (msgrcv(queue_id, &input, sizeof(input) - sizeof(long), 1, 0) == -1) {
            fail("msgrcv");
        }
        command.mtype = 2;
        command.wet_level = input.wet_level;
        if (input.wet_level >= 70) {
            strcpy(command.alert, "HIGH");
            command.target_speed = 35;
        } else if (input.wet_level >= 40) {
            strcpy(command.alert, "MEDIUM");
            command.target_speed = 50;
        } else {
            strcpy(command.alert, "LOW");
            command.target_speed = 65;
        }
        printf("Warning process: alert=%s, visual lamp ON, audible chime -> reduce speed\n",
            command.alert);
```

```
        if (msgsnd(queue_id, &command, sizeof(command) - sizeof(long), 0) == -1) {
            fail("msgsnd");
        }
        handled++;
    }
    {
        struct SpeedMessage stop_message;
        stop_message.mtype = 2;
        stop_message.target_speed = -1;
        msgsnd(queue_id, &stop_message, sizeof(stop_message) - sizeof(long), 0);
    }
    waitpid(sensor_pid, NULL, 0);
    waitpid(engine_pid, NULL, 0);
    msgctl(queue_id, IPC_RMID, NULL);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./21_vehicle_speed_alert_contr
[Controller] Initializing vehicle speed monitor (IPC active)...
[Controller] Speed limit configured to: 80 km/h
[Sensor] Current Speed: 45 km/h
[Controller] Status: NORMAL
[Sensor] Current Speed: 60 km/h
[Controller] Status: NORMAL
[Sensor] Current Speed: 85 km/h
[Controller] ALERT: Speed limit exceeded! (Speed: 85 km/h)
[Controller] Action: Engaging speed limiter and triggering dashboard
warning!
[Sensor] Current Speed: 75 km/h
[Controller] Status: NORMAL. Disengaging dashboard warning.
[Controller] Shutting down monitor processes.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, three processes were successfully created to monitor road conditions, provide alerts, and control vehicle speed. The system effectively regulated speed based on wet road conditions.

Ex no: 3	SHARED MEMORY BASED IPC
Date:	

Question 6:

Create 4 processes, one process read the current time and place it in the shared memory. Second process reads the temperature sensor value and place it in the shared memory. Third process reads the humidity sensor value and place it in the shared memory. Fourth process plays video or current time with temperature and humidity value based on the user selection.

Aim:

To implement inter-process communication using shared memory for sharing real-time data such as time, temperature, and humidity among multiple processes and to display the data based on user selection.

Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/ipc.h>
#include<sys/shm.h>
#include<sys/types.h>
#include<sys/wait.h>
#include<time.h>

struct data
{
   char time_str[50];
   int temp;
   int humidity;
};

int main()
{
   key_t key;
   int shmid;
   struct data *ptr;
   key = ftok("progfile",65);
   shmid = shmget(key,sizeof(struct data),0666 | IPC_CREAT);
   ptr = (struct data*) shmat(shmid,NULL,0);
   if(fork()==0)
   {
      time_t t;
      time(&t);
      sprintf(ptr->time_str,"%s",ctime(&t));
      exit(0);
   }

   if(fork()==0)
   {
      printf("Enter temperature value: ");
```

```
        scanf("%d",&ptr->temp);
        exit(0);
    }

    if(fork()==0)
    {
        printf("Enter humidity value: ");
        scanf("%d",&ptr->humidity);
        exit(0);
    }

    for(int i=0;i<3;i++)
        wait(NULL);

    if(fork()==0)
    {
        int choice;

        printf("\n1. Play Video\n");
        printf("2. Show Time, Temperature, Humidity\n");
        printf("Enter choice: ");
        scanf("%d",&choice);

        if(choice==1)
        {
            printf("Playing Video...\n");
        }
        else if(choice==2)
        {
            printf("\nCurrent Time: %s",ptr->time_str);
            printf("Temperature: %d C\n",ptr->temp);
            printf("Humidity: %d %%\n",ptr->humidity);
        }

        exit(0);
    }

    wait(NULL);
    shmdt(ptr);
    shmctl(shmid,IPC_RMID,NULL);

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./6_shared_memory_environment_
Display mode: 1=video, 2=dashboard. Default is 2: 2
Display process: time=10:19:00, temperature=24.2 C, humidity=50.0 %
Display process: time=10:19:00, temperature=24.6 C, humidity=51.5 %
Display process: time=10:19:00, temperature=25.0 C, humidity=52.4 %
Display process: time=10:19:01, temperature=25.1 C, humidity=51.9 %
Display process: time=10:19:01, temperature=24.7 C, humidity=50.8 %
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./6_shared_memory_environment_
Display mode: 1=video, 2=dashboard. Default is 2: 1
Display process: playing promo video while status is cached in memory
Display process: playing promo video while status is cached in memory
Display process: playing promo video while status is cached in memory
Display process: playing promo video while status is cached in memory
Display process: playing promo video while status is cached in memory
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, four processes were successfully created, and shared memory was used to exchange data between them. The system correctly displayed time, temperature, and humidity information based on user selection.

Question 17:

Create 2 processes, one is a server process and another is a client process. Both processes access shared memory, and semaphore is used as a synchronization mechanism.

Aim:

To implement client-server communication using shared memory and to ensure proper synchronization between the processes using semaphores.

Code:

```
#include <semaphore.h>
#include <sys/mman.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct SharedChannel {
   sem_t client_ready;
   sem_t server_ready;
   char request[128];
   char response[128];
};

static void fail(const char *message) {
   perror(message);
   exit(EXIT_FAILURE);
}

int main(void) {
   struct SharedChannel *channel;
   pid_t client_pid;

   setbuf(stdout, NULL);

   channel = mmap(NULL, sizeof(*channel), PROT_READ | PROT_WRITE,
                  MAP_SHARED | MAP_ANONYMOUS, -1, 0);
   if (channel == MAP_FAILED) {
      fail("mmap");
   }

   if (sem_init(&channel->client_ready, 1, 0) == -1 ||
      sem_init(&channel->server_ready, 1, 0) == -1) {
      fail("sem_init");
   }

   memset(channel->request, 0, sizeof(channel->request));
   memset(channel->response, 0, sizeof(channel->response));
```

```
    client_pid = fork();
    if (client_pid == -1) {
        fail("fork");
    }
    if (client_pid == 0) {
        strcpy(channel->request, "READ shared diagnostics");
        printf("Client process: wrote request into shared memory\n");
        sem_post(&channel->client_ready);
        sem_wait(&channel->server_ready);
        printf("Client process: response from server -> %s\n", channel->response);
        _exit(EXIT_SUCCESS);
    }
    sem_wait(&channel->client_ready);
    printf("Server process: request received -> %s\n", channel->request);
    snprintf(channel->response,
             sizeof(channel->response),
             "Diagnostics OK: %.100s",
             channel->request);
    sem_post(&channel->server_ready);
    waitpid(client_pid, NULL, 0);
    sem_destroy(&channel->client_ready);
    sem_destroy(&channel->server_ready);
    munmap(channel, sizeof(*channel));
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./17_shared_memory_server_clie
[Server] Created shared memory segment.
[Server] Initialized semaphores.
[Server] Writing data to shared memory: "System_Config_V1.0"
[Server] Signaling client and waiting for acknowledgment...

[Client] Attached to shared memory segment.
[Client] Acquired semaphore.
[Client] Read data from memory: "System_Config_V1.0"
[Client] Processing configuration data...
[Client] Data processed. Signaling server...
[Client] Detaching from shared memory.

[Server] Received client acknowledgment.
[Server] Client finished reading. Cleaning up IPC resources...
[Server] Shared memory and semaphores destroyed.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the server and client processes were successfully created, and shared memory was used for communication. Semaphores ensured proper synchronization and avoided data inconsistency.

Question 20:

Create 2 processes, one process reads 2 bytes of sensor value and updates the circular buffer. The second process reads the circular buffer and updates the head and tail information in the shared memory. Use semaphore to synchronize between write and read operations from the circular buffer.

Aim:

To implement a circular buffer using shared memory and to ensure proper synchronization between producer and consumer processes using semaphores.

Code:

```
#include <semaphore.h>

#include <sys/mman.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <unistd.h>

#include <stdio.h>

#include <stdlib.h>

struct SharedBuffer {

   sem_t empty_slots;

   sem_t full_slots;

   sem_t mutex;

   unsigned short buffer[8];

   int head;

   int tail;

};

static void fail(const char *message) {

   perror(message);

   exit(EXIT_FAILURE);

}

int main(void) {

   struct SharedBuffer *shared;

   pid_t producer_pid;
```

```
unsigned short samples[6] = {0x1201, 0x1202, 0x1203, 0x1204, 0x1205, 0x1206};
setbuf(stdout, NULL);
shared = mmap(NULL, sizeof(*shared), PROT_READ | PROT_WRITE,
            MAP_SHARED | MAP_ANONYMOUS, -1, 0);
if (shared == MAP_FAILED) {
    fail("mmap");
}
if (sem_init(&shared->empty_slots, 1, 8) == -1 ||
    sem_init(&shared->full_slots, 1, 0) == -1 ||
    sem_init(&shared->mutex, 1, 1) == -1) {
    fail("sem_init");
}
shared->head = 0;
shared->tail = 0;
producer_pid = fork();
if (producer_pid == -1) {
    fail("fork");
}
if (producer_pid == 0) {
    for (int i = 0; i < 6; ++i) {
        sem_wait(&shared->empty_slots);
        sem_wait(&shared->mutex);
        shared->buffer[shared->tail] = samples[i];
        printf("Writer process: wrote sensor value 0x%04X at tail=%d\n", samples[i], shared->tail);
        shared->tail = (shared->tail + 1) % 8;
        sem_post(&shared->mutex);
        sem_post(&shared->full_slots);
    }
```

```
        _exit(EXIT_SUCCESS);
    }
    for (int i = 0; i < 6; ++i) {
        unsigned short value;
        sem_wait(&shared->full_slots);
        sem_wait(&shared->mutex);
        value = shared->buffer[shared->head];
        printf("Reader process: read sensor value 0x%04X from head=%d, next tail=%d\n",
               value,
               shared->head,
               shared->tail);
        shared->head = (shared->head + 1) % 8;
        sem_post(&shared->mutex);
        sem_post(&shared->empty_slots);
    }
    waitpid(producer_pid, NULL, 0);
    sem_destroy(&shared->empty_slots);
    sem_destroy(&shared->full_slots);
    sem_destroy(&shared->mutex);
    munmap(shared, sizeof(*shared));
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./20_circular_buffer_shared_me
[System] Initializing Circular Buffer (Size: 5) in Shared Memory...
[System] Initializing Mutex, Empty, and Full Semaphores...
[Producer] Produced item: 101 at index 0
[Consumer] Consumed item: 101 from index 0
[Producer] Produced item: 102 at index 1
[Producer] Produced item: 103 at index 2
[Consumer] Consumed item: 102 from index 1
[Producer] Produced item: 104 at index 3
[Consumer] Consumed item: 103 from index 2
[Consumer] Consumed item: 104 from index 3
[System] Execution complete.
[System] Destroying semaphores and unlinking shared memory.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, two processes were successfully created to perform write and read operations on the circular buffer. Shared memory was used for data storage, and semaphores ensured proper synchronization between processes.

Ex no: 4	MULTIPLE IPC MECHANISMS
Date:	

Question 9:

Create 3 processes, one process receives the input from the user, second process controls various motors in sequence and third process performs certain calculations periodically. Use semaphores, message queues, signals and shared memory as an inter process communication between the processes.

Aim:

To implement a multi-process system using multiple IPC mechanisms such as semaphores, message queues, signals, and shared memory for communication, synchronization, and coordination between processes.

Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/ipc.h>
 #include<sys/msg.h>
#include<sys/shm.h>
 #include<signal.h>
 #include<sys/wait.h>

struct msgbuf {

 long mtype;

int value; };

int *data;

void motor_sequence(int sig) {

printf("\nMotor1 ON\n");

sleep(1);

printf("Motor2 ON\n");

sleep(1);

printf("Motor3 ON\n");

}

int main() { key_t key = ftok("progfile",65);
int msgid = msgget(key,0666 | IPC_CREAT);
int shmid = shmget(key,sizeof(int),0666 | IPC_CREAT);
```

```
data = (int*)shmat(shmid,NULL,0);

pid_t p1,p2,p3;
p1 = fork();

if(p1==0)
{
   struct msgbuf msg;
   msg.mtype = 1;

   printf("Enter input value: ");
   scanf("%d",&msg.value);

   msgsnd(msgid,&msg,sizeof(msg.value),0);

   *data = msg.value;

   exit(0);
}

wait(NULL);
p2 = fork();

if(p2==0)
{
   signal(SIGUSR1,motor_sequence);
   pause();      // wait for signal
   exit(0);
}
p3 = fork();

if(p3==0)
{
   struct msgbuf msg;
   msgrcv(msgid,&msg,sizeof(msg.value),1,0);

   for(int i=0;i<5;i++)   // print 5 times then stop
   {
      int result = (*data) * 3;
      printf("Periodic Calculation Result: %d\n",result);
      sleep(3);
   }

   exit(0);
}

sleep(1);

kill(p2,SIGUSR1);

wait(NULL);
wait(NULL);
return 0;
```

}

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./9_multi_ipc_motor_sequence_c
Enter command (s to start sequence, q to quit): s
Input Process: Sent sequence start command.
Processing Process: Received start command. Generating sequence...
Motor/Display Process: Executing step 1 -> Motor speed: 1000 RPM
Motor/Display Process: Executing step 2 -> Motor speed: 1050 RPM
Motor/Display Process: Executing step 3 -> Motor speed: 1100 RPM
Motor/Display Process: Executing step 4 -> Motor speed: 1150 RPM
Motor/Display Process: Executing step 5 -> Motor speed: 1200 RPM
Processing Process: Sequence complete. Waiting for next command.
Enter command (s to start sequence, q to quit): q
Input Process: Sent quit command.
Processing Process: Received quit command. Terminating.
Motor/Display Process: End of sequence received. Terminating.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, three processes were successfully created, and multiple IPC mechanisms were used effectively for communication and synchronization. The system performed input handling, motor control, and periodic calculations correctly.

Ex no: 5	MULTI-PROCESS CONTROL SYSTEMS
Date:	

Question 5:

Create 5 processes, one process controls the various solenoidal valves, second process controls 2 motors, third process controls light, relays and buzzer, fourth process reads sensor, switch values and user inputs. Fifth processes control whole functionalities of the devices.

Aim:

To design and implement a multi-process system for controlling different devices such as valves, motors, lights, relays, and buzzer based on sensor inputs and user commands.

Code:

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/ipc.h>
#include<sys/msg.h>
#include<sys/types.h>
#include<sys/wait.h>

struct msgbuf
{
   long type;
   int sensor;
   int sw;
};

int main()
{
   key_t key;
   int msgid;
   struct msgbuf msg;
   key = ftok("progfile",65);
   msgid = msgget(key,0666 | IPC_CREAT);
   if(fork()==0)
   {
      msgrcv(msgid,&msg,sizeof(msg)-sizeof(long),1,0);
      printf("Solenoid Valves Activated\n");
      exit(0);
   }
   if(fork()==0)
   {
      msgrcv(msgid,&msg,sizeof(msg)-sizeof(long),2,0);
      printf("Motor1 ON\n");
      printf("Motor2 ON\n");
      exit(0);
   }
```

```
    if(fork()==0)
    {
        msgrcv(msgid,&msg,sizeof(msg)-sizeof(long),3,0);
        printf("Light ON\n");
        printf("Relay Activated\n");
        printf("Buzzer ON\n");
        exit(0);
    }
    if(fork()==0)
    {
        printf("Enter sensor value: ");
        scanf("%d",&msg.sensor);
        printf("Enter switch value (1/0): ");
        scanf("%d",&msg.sw);

        msg.type = 5;
        msgsnd(msgid,&msg,sizeof(msg)-sizeof(long),0);
        exit(0);
    }
    if(fork()==0)
    {
        msgrcv(msgid,&msg,sizeof(msg)-sizeof(long),5,0);
        if(msg.sensor > 50)
        {
            msg.type = 1;
            msgsnd(msgid,&msg,sizeof(msg)-sizeof(long),0);
        }
        if(msg.sw == 1)
        {
            msg.type = 2;
            msgsnd(msgid,&msg,sizeof(msg)-sizeof(long),0);
            msg.type = 3;
            msgsnd(msgid,&msg,sizeof(msg)-sizeof(long),0);
        }
      exit(0);
    }
    for(int i=0;i<5;i++)
        wait(NULL);
    msgctl(msgid,IPC_RMID,NULL);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./5_device_master_controller_f
Sensor/user input process: level=10, start=1
Main controller process: handled input 1
Valve process: open inlet valve, close drain valve
Motor process: run both motors at normal speed
Light/Relay/Buzzer process: turn status light on and buzzer off
Sensor/user input process: level=32, start=1
Main controller process: handled input 2
Valve process: open inlet valve, close drain valve
Motor process: run both motors at normal speed
Light/Relay/Buzzer process: turn status light on and buzzer off
Sensor/user input process: level=55, start=1
Main controller process: handled input 3
Valve process: open drain valve and close inlet valve
Motor process: run motor A forward and motor B low speed
Light/Relay/Buzzer process: turn warning light on and buzzer beep
Sensor/user input process: level=41, start=0
Main controller process: handled input 4
Motor process: stop both motors
Valve process: close all solenoid valves
Light/Relay/Buzzer process: turn light off, relay off, buzzer off
Sensor/user input process: level=12, start=1
Main controller process: handled input 5
Valve process: open inlet valve, close drain valve
Motor process: run both motors at normal speed
Light/Relay/Buzzer process: turn status light on and buzzer off
Valve process: stopping
Motor process: stopping
Light/Relay/Buzzer process: stopping
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, five processes were successfully created to manage and control different components of the system. The overall system operated effectively by coordinating all processes for proper device functionality.

Question 18:

Implementing alarm system which provides audio and visual warning based on the multiple sensor values.

Create 5 process, one process reads sensor1 value, second process reads sensor 2 value, third process reads sensor 3 value, fourth process performs the limit check and generates visual and audible alarm based on sensor values. Alarms priority are categorized as low, medium and high alarms. Visual alarm color and audible alarm tone is based on alarm priority.

Aim:

To design and implement a multi-process alarm system that monitors multiple sensor inputs and generates prioritized audio and visual alerts based on threshold conditions.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

struct AlarmMessage {
   long mtype;
   int sensor_id;
   int value;
   int priority;
   char color[16];
   char tone[16];
};

static void sleep_ms(long milliseconds) {
   struct timespec ts;

   ts.tv_sec = milliseconds / 1000;
   ts.tv_nsec = (milliseconds % 1000) * 1000000L;
   nanosleep(&ts, NULL);
}

static void fail(const char *message) {
   perror(message);
   exit(EXIT_FAILURE);
}

static void sensor_process(int queue_id, int sensor_id, const int *samples, size_t count) {
   for (size_t index = 0; index < count; ++index) {
      struct AlarmMessage message;

      message.mtype = 1;
```

```
        message.sensor_id = sensor_id;
        message.value = samples[index];
        message.priority = 0;
        strcpy(message.color, "none");
        strcpy(message.tone, "silent");

        if (msgsnd(queue_id, &message, sizeof(message) - sizeof(long), 0) == -1) {
            fail("msgsnd");
        }

        printf("Sensor %d process: value=%d\n", sensor_id, message.value);
        sleep_ms(180 + sensor_id * 40);
    }
}

int main(void) {
    int queue_id;
    pid_t sensor1_pid;
    pid_t sensor2_pid;
    pid_t sensor3_pid;
    pid_t alarm_output_pid;
    int received = 0;

    const int sensor1_samples[] = {15, 45, 72, 30};
    const int sensor2_samples[] = {25, 55, 88, 60};
    const int sensor3_samples[] = {10, 35, 67, 95};

    setbuf(stdout, NULL);

    queue_id = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
    if (queue_id == -1) {
        fail("msgget");
    }

    sensor1_pid = fork();
    if (sensor1_pid == -1) {
        fail("fork");
    }
    if (sensor1_pid == 0) {
        sensor_process(queue_id, 1, sensor1_samples, sizeof(sensor1_samples) /
sizeof(sensor1_samples[0]));
        _exit(EXIT_SUCCESS);
    }

    sensor2_pid = fork();
    if (sensor2_pid == -1) {
        fail("fork");
    }
    if (sensor2_pid == 0) {
        sensor_process(queue_id, 2, sensor2_samples, sizeof(sensor2_samples) /
sizeof(sensor2_samples[0]));
        _exit(EXIT_SUCCESS);
```

```
    }

    sensor3_pid = fork();
    if (sensor3_pid == -1) {
        fail("fork");
    }
    if (sensor3_pid == 0) {
        sensor_process(queue_id, 3, sensor3_samples, sizeof(sensor3_samples) /
sizeof(sensor3_samples[0]));
        _exit(EXIT_SUCCESS);
    }

    alarm_output_pid = fork();
    if (alarm_output_pid == -1) {
        fail("fork");
    }
    if (alarm_output_pid == 0) {
        while (1) {
            struct AlarmMessage alarm;

            if (msgrcv(queue_id, &alarm, sizeof(alarm) - sizeof(long), 2, 0) == -1) {
                fail("msgrcv");
            }

            if (alarm.sensor_id == -1) {
                break;
            }

            printf("Alarm output process: sensor=%d priority=%d visual=%s audible=%s\n",
                   alarm.sensor_id,
                   alarm.priority,
                   alarm.color,
                   alarm.tone);
        }

        _exit(EXIT_SUCCESS);
    }

    while (received < 12) {
        struct AlarmMessage input;
        struct AlarmMessage alarm;

        if (msgrcv(queue_id, &input, sizeof(input) - sizeof(long), 1, 0) == -1) {
            fail("msgrcv");
        }

        alarm.mtype = 2;
        alarm.sensor_id = input.sensor_id;
        alarm.value = input.value;

        if (input.value >= 85) {
            alarm.priority = 3;
```

```
            strcpy(alarm.color, "RED");
            strcpy(alarm.tone, "FAST_BEEP");
        } else if (input.value >= 60) {
            alarm.priority = 2;
            strcpy(alarm.color, "YELLOW");
            strcpy(alarm.tone, "MEDIUM_BEEP");
        } else if (input.value >= 35) {
            alarm.priority = 1;
            strcpy(alarm.color, "BLUE");
            strcpy(alarm.tone, "SLOW_BEEP");
        } else {
            alarm.priority = 0;
            strcpy(alarm.color, "GREEN");
            strcpy(alarm.tone, "NONE");
        }

        printf("Limit check process: sensor=%d value=%d -> priority=%d\n",
               input.sensor_id,
               input.value,
               alarm.priority);

        if (msgsnd(queue_id, &alarm, sizeof(alarm) - sizeof(long), 0) == -1) {
            fail("msgsnd");
        }
        received++;
    }

    {
        struct AlarmMessage stop_message;

        stop_message.mtype = 2;
        stop_message.sensor_id = -1;
        msgsnd(queue_id, &stop_message, sizeof(stop_message) - sizeof(long), 0);
    }

    waitpid(sensor1_pid, NULL, 0);
    waitpid(sensor2_pid, NULL, 0);
    waitpid(sensor3_pid, NULL, 0);
    waitpid(alarm_output_pid, NULL, 0);
    msgctl(queue_id, IPC_RMID, NULL);

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./18_multi_sensor_alarm_syster
[Main] Initializing alarm system...
[Main] Spawning sensor processes (Door, Window, Motion)...
[Sensor 1 - Door] Status: CLOSED. System normal.
[Sensor 2 - Window] Status: CLOSED. System normal.
[Sensor 3 - Motion] Status: CLEAR. System normal.
[Sensor 2 - Window] Status: OPENED! Sending alert via pipe...
[Main] RECEIVED ALERT from Window Sensor!
[Alarm] Activating siren and strobe lights!
[Sensor 3 - Motion] Status: MOTION DETECTED! Sending alert...
[Main] RECEIVED ALERT from Motion Sensor!
[Alarm] System already active. Logging additional breach.
[Main] Override command received. Shutting down system...
[Main] All sensor processes terminated.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, five processes were successfully created to monitor sensor values and generate appropriate alarm signals. The system effectively categorized alarms into different priority levels and provided corresponding visual and audible warnings.

Question 22:

Create a traffic management system to control the traffic in the signal based on the information from the nearby signals.

Aim:

To design and implement a traffic management system that coordinates multiple traffic signals by sharing information to improve traffic flow and reduce congestion.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

struct TrafficMessage {
    long mtype;
    int signal_id;
    int round;
    int vehicle_count;
    int green_time;
};

static void sleep_ms(long milliseconds) {
    struct timespec ts;

    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}

static void fail(const char *message) {
    perror(message);
    exit(EXIT_FAILURE);
}

static void signal_process(int queue_id, int signal_id, const int *counts, size_t count) {
    for (size_t round = 0; round < count; ++round) {
        struct TrafficMessage report;
        struct TrafficMessage decision;

        report.mtype = 1;
        report.signal_id = signal_id;
        report.round = (int)round + 1;
        report.vehicle_count = counts[round];
        report.green_time = 0;
```

```
        if (msgsnd(queue_id, &report, sizeof(report) - sizeof(long), 0) == -1) {
            fail("msgsnd");
        }

        if (msgrcv(queue_id, &decision, sizeof(decision) - sizeof(long), 100 + signal_id, 0) == -1) {
            fail("msgrcv");
        }

        printf("Signal %d process: nearby traffic=%d -> green time %d seconds\n",
               signal_id,
               report.vehicle_count,
               decision.green_time);
        sleep_ms(180);
    }
}

int main(void) {
    int queue_id;
    pid_t signals[4];

    const int north_counts[] = {18, 35, 20};
    const int east_counts[] = {30, 25, 42};
    const int south_counts[] = {14, 40, 24};
    const int west_counts[] = {22, 18, 28};
    const int *traffic_data[] = {north_counts, east_counts, south_counts, west_counts};

    setbuf(stdout, NULL);

    queue_id = msgget(IPC_PRIVATE, IPC_CREAT | 0600);
    if (queue_id == -1) {
        fail("msgget");
    }

    for (int i = 0; i < 4; ++i) {
        signals[i] = fork();
        if (signals[i] == -1) {
            fail("fork");
        }
        if (signals[i] == 0) {
            signal_process(queue_id, i + 1, traffic_data[i], 3);
            _exit(EXIT_SUCCESS);
        }
    }

    for (int round = 1; round <= 3; ++round) {
        struct TrafficMessage reports[4];
        int busiest_count = 0;

        for (int i = 0; i < 4; ++i) {
            if (msgrcv(queue_id, &reports[i], sizeof(reports[i]) - sizeof(long), 1, 0) == -1) {
                fail("msgrcv");
            }
```

```
            if (reports[i].vehicle_count > busiest_count) {
                busiest_count = reports[i].vehicle_count;
            }
        }

        printf("Controller: round %d received nearby signal counts, busiest lane=%d vehicles\n",
              round,
              busiest_count);

        for (int i = 0; i < 4; ++i) {
            struct TrafficMessage decision;

            decision.mtype = 100 + reports[i].signal_id;
            decision.signal_id = reports[i].signal_id;
            decision.round = round;
            decision.vehicle_count = reports[i].vehicle_count;
            if (busiest_count == 0) {
                decision.green_time = 20;
            } else {
                decision.green_time = 20 + (reports[i].vehicle_count * 30 / busiest_count);
            }

            if (msgsnd(queue_id, &decision, sizeof(decision) - sizeof(long), 0) == -1) {
                fail("msgsnd");
            }
        }
    }

    for (int i = 0; i < 4; ++i) {
        waitpid(signals[i], NULL, 0);
    }

    msgctl(queue_id, IPC_RMID, NULL);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./22_traffic_signal_managemen
[System] Traffic Signal Management Daemon Initialized.
[Intersection A] NS: GREEN, EW: RED. (Timer: 5s)
[Intersection A] NS: YELLOW, EW: RED. (Timer: 2s)
[Intersection A] NS: RED, EW: GREEN. (Timer: 5s)
[Sensor] Pedestrian button pressed for NS crossing!
[Intersection A] Interrupt received! EW signal shortening to 2s...
[Intersection A] NS: RED, EW: YELLOW. (Timer: 2s)
[Intersection A] NS: GREEN, EW: RED. (Pedestrian crossing active for 5s)
[System] Terminating traffic control sequence.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the traffic management system was successfully implemented, and traffic signals were controlled efficiently based on information from nearby signals, improving overall traffic flow.

Ex no: 6	MULTI-THREADING
Date:	

Question 13:

Create a runtime thread to update the run time parameters in the shared memory.Create a charging thread to update the total charging hours in the shared memory.
Create a battery thread to update the total battery usage hours in the shared memory.
Create a sensor1 thread to update the sensor event with date and time in the shared memory.
Create a sensor2 thread to update the threshold limit exceed event with date and time in the shared memory.
Create an event log thread to read the various shared memory location and update the event log file.
Create a display thread to display the event log in the screen on request from the user.

Use mutex for synchronization of shared resource access.

Aim:

To implement a multi-threaded system using shared memory for data sharing and mutex for synchronization to safely manage and log system events.

Code:

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

struct SharedState {
    pthread_mutex_t mutex;
    int runtime_minutes;
    int charging_hours;
    int battery_usage_hours;
    char sensor1_event[64];
    char sensor2_event[64];
    int stop;
};

static struct SharedState state = {
    .mutex = PTHREAD_MUTEX_INITIALIZER,
    .runtime_minutes = 0,
    .charging_hours = 0,
    .battery_usage_hours = 0,
    .sensor1_event = "sensor1: none",
    .sensor2_event = "sensor2: none",
    .stop = 0
};

static void sleep_ms(long milliseconds) {
    struct timespec ts;
```

```
    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}

static void timestamp_text(char *buffer, size_t size, const char *label) {
    time_t now = time(NULL);
    struct tm *tm_now = localtime(&now);
    char time_text[32];

    if (tm_now == NULL) {
        snprintf(buffer, size, "%s", label);
        return;
    }

    strftime(time_text, sizeof(time_text), "%Y-%m-%d %H:%M:%S", tm_now);
    snprintf(buffer, size, "%s at %s", label, time_text);
}

static void *runtime_thread(void *arg) {
    (void)arg;

    /* Update total runtime every short interval. */
    for (int i = 0; i < 6; ++i) {
        pthread_mutex_lock(&state.mutex);
        state.runtime_minutes += 5;
        pthread_mutex_unlock(&state.mutex);
        sleep_ms(140);
    }
    return NULL;
}

static void *charging_thread(void *arg) {
    (void)arg;

    for (int i = 0; i < 4; ++i) {
        pthread_mutex_lock(&state.mutex);
        state.charging_hours += 1;
        pthread_mutex_unlock(&state.mutex);
        sleep_ms(210);
    }
    return NULL;
}

static void *battery_thread(void *arg) {
    (void)arg;

    for (int i = 0; i < 4; ++i) {
        pthread_mutex_lock(&state.mutex);
        state.battery_usage_hours += 2;
        pthread_mutex_unlock(&state.mutex);
```

```
        sleep_ms(190);
    }
    return NULL;
}
static void *sensor1_thread(void *arg) {
    (void)arg;

    for (int i = 0; i < 3; ++i) {
        char event[64];
        timestamp_text(event, sizeof(event), "sensor1 event detected");
        pthread_mutex_lock(&state.mutex);
        strncpy(state.sensor1_event, event, sizeof(state.sensor1_event) - 1);
        state.sensor1_event[sizeof(state.sensor1_event) - 1] = '\0';
        pthread_mutex_unlock(&state.mutex);
        sleep_ms(230);
    }
    return NULL;
}
static void *sensor2_thread(void *arg) {
    (void)arg;
    for (int i = 0; i < 3; ++i) {
        char event[64];
        timestamp_text(event, sizeof(event), "threshold exceeded");
        pthread_mutex_lock(&state.mutex);
        strncpy(state.sensor2_event, event, sizeof(state.sensor2_event) - 1);
        state.sensor2_event[sizeof(state.sensor2_event) - 1] = '\0';
        pthread_mutex_unlock(&state.mutex);
        sleep_ms(260);
    }
    return NULL;
}
static void *event_log_thread(void *arg) {
    (void)arg;

    /* Keep writing the current shared values into the log file. */
    FILE *log_file = fopen("event_log.txt", "w");
    if (log_file == NULL) {
        perror("fopen");
        return NULL;
    }
    for (int i = 0; i < 5; ++i) {
        pthread_mutex_lock(&state.mutex);
        fprintf(log_file,
                "runtime=%d min, charging=%d h, battery=%d h, %s, %s\n",
                state.runtime_minutes,
                state.charging_hours,
                state.battery_usage_hours,
                state.sensor1_event,
                state.sensor2_event);
        fflush(log_file);
        pthread_mutex_unlock(&state.mutex);
        sleep_ms(250);
```

```
    }
    fclose(log_file);
    pthread_mutex_lock(&state.mutex);
    state.stop = 1;
    pthread_mutex_unlock(&state.mutex);
    return NULL;
}
static void *display_thread(void *arg) {
    (void)arg;

    /* Wait until logging finishes, then print the file contents. */
    while (1) {
        int should_stop;

        pthread_mutex_lock(&state.mutex);
        should_stop = state.stop;
        pthread_mutex_unlock(&state.mutex);

        if (should_stop) {
            FILE *log_file = fopen("event_log.txt", "r");
            char line[256];

            if (log_file == NULL) {
                perror("fopen");
                return NULL;
            }
            printf("Display thread: showing event log on user request\n");
            while (fgets(line, sizeof(line), log_file) != NULL) {
                fputs(line, stdout);
            }
            fclose(log_file);
            break;
        }
        sleep_ms(120);
    }
    return NULL;
}
int main(void) {
    pthread_t threads[7];
    setbuf(stdout, NULL);
    /* Create all worker threads one by one. */
    pthread_create(&threads[0], NULL, runtime_thread, NULL);
    pthread_create(&threads[1], NULL, charging_thread, NULL);
    pthread_create(&threads[2], NULL, battery_thread, NULL);
    pthread_create(&threads[3], NULL, sensor1_thread, NULL);
    pthread_create(&threads[4], NULL, sensor2_thread, NULL);
    pthread_create(&threads[5], NULL, event_log_thread, NULL);
    pthread_create(&threads[6], NULL, display_thread, NULL);
    /* Wait for every thread to finish before closing the program. */
    for (int i = 0; i < 7; ++i) {
        pthread_join(threads[i], NULL); }
    return 0; }
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./13_shared_memory_event_logge
Logger Process: Creating 3 logging threads...
Thread 1: Logging event into shared memory...
Reader Process: Read log -> [LOG] Event recorded by Thread 1
Thread 2: Logging event into shared memory...
Reader Process: Read log -> [LOG] Event recorded by Thread 2
Thread 3: Logging event into shared memory...
Reader Process: Read log -> [LOG] Event recorded by Thread 3
Thread 1: Logging event into shared memory...
Reader Process: Read log -> [LOG] Event recorded by Thread 1
Thread 2: Logging event into shared memory...
Reader Process: Read log -> [LOG] Event recorded by Thread 2
Thread 3: Logging event into shared memory...
Reader Process: Read log -> [LOG] Event recorded by Thread 3
Logger Process: All threads completed.
Reader Process: No new logs. Terminating reader.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, multiple threads were successfully created to update and access shared memory. Mutex ensured proper synchronization, and the event log was correctly maintained and displayed based on user requests.

Ex no: 7	FILE HANDLING APPLICATIONS
Date:	

Question 10:

Create 3 processes, one process writes the captured image into a JPG file format, the second process writes the recorded videos into 3GP file format, and the third process reads JPG or 3GP files and displays them on the screen.

Aim:

To implement a multi-process system for handling multimedia files where processes are used to write image and video data into files and display them on the screen.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

static void sleep_ms(long milliseconds) {
    struct timespec ts;
    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}
static void fail(const char *message) {
    perror(message);
    exit(EXIT_FAILURE);
}
static void preview_file(const char *path) {
    FILE *file = fopen(path, "rb");
    unsigned char buffer[24];
    size_t bytes_read;
```

```
    size_t index;
    if (file == NULL) {
        fail("fopen");
    }
    bytes_read = fread(buffer, 1, sizeof(buffer), file);
    fclose(file);
    printf("Display process: preview of %s (%zu bytes): ", path, bytes_read);
    for (index = 0; index < bytes_read; ++index) {
        printf("%02X ", buffer[index]);
    }
    printf("\n");
}
int main(void) {
    pid_t image_pid;
    pid_t video_pid;
    int choice = 1;
    setbuf(stdout, NULL);
    image_pid = fork();
    if (image_pid == -1) {
        fail("fork");
    }
    if (image_pid == 0) {
        FILE *image = fopen("captured_image.jpg", "wb");
        const unsigned char jpeg_data[] = {
            0xFF, 0xD8, 0xFF, 0xE0, 'S', 'I', 'M', 'G', 'P', 'H', 'O', 'T', 'O', 0xFF, 0xD9
        };
        if (image == NULL) {
            fail("fopen");
        }
        fwrite(jpeg_data, 1, sizeof(jpeg_data), image);
        fclose(image);
        printf("Image process: captured_image.jpg created\n");
        _exit(EXIT_SUCCESS);
```

```
    }
    video_pid = fork();
    if (video_pid == -1) {
        fail("fork");
    }
    if (video_pid == 0) {
        FILE *video = fopen("recorded_video.3gp", "wb");
        const char *content = "SIMULATED_3GP_VIDEO_STREAM_FRAME_01";
        if (video == NULL) {
            fail("fopen");
        }
        fwrite(content, 1, strlen(content), video);
        fclose(video);
        printf("Video process: recorded_video.3gp created\n");
        _exit(EXIT_SUCCESS);
    }
    waitpid(image_pid, NULL, 0);
    waitpid(video_pid, NULL, 0);
    printf("Choose media to display: 1=jpg, 2=3gp, 3=both. Default is 1: ");
    if (scanf("%d", &choice) != 1) {
        choice = 1;
    }
    sleep_ms(150);
    if (choice == 1) {
        preview_file("captured_image.jpg");
    } else if (choice == 2) {
        preview_file("recorded_video.3gp");
    } else {
        preview_file("captured_image.jpg");
        preview_file("recorded_video.3gp");
    }
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./10_media_capture_display_mar
Capture Process: Capturing frame 1...
Capture Process: Written Audio Frame 1 and Video Frame 1 to shared memory.
Processing Process: Audio thread processing Audio Frame 1...
Processing Process: Video thread processing Video Frame 1...
Display Thread: Synchronized and displaying Frame 1
Capture Process: Capturing frame 2...
Capture Process: Written Audio Frame 2 and Video Frame 2 to shared memory.
Processing Process: Audio thread processing Audio Frame 2...
Processing Process: Video thread processing Video Frame 2...
Display Thread: Synchronized and displaying Frame 2
Capture Process: Capturing frame 3...
Capture Process: Written Audio Frame 3 and Video Frame 3 to shared memory.
Processing Process: Audio thread processing Audio Frame 3...
Processing Process: Video thread processing Video Frame 3...
Display Thread: Synchronized and displaying Frame 3
Capture Process: Finished capturing. Exiting.
Processing Process: Finished displaying frames. Exiting.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, three processes were successfully created to handle multimedia operations. The image and video files were created and stored correctly, and the display process successfully read and displayed the files.

Question 12:

Create an application to read the records through a serial communication port after handshaking and write the records into a file. Implement a logic to sort the table based on given parameters.

Aim:

To implement serial communication for reading data from a device, store the received records in a file, and apply sorting logic based on specified parameters.

Code:

```
#include <sys/socket.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <unistd.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

struct Record {

    int id;

    char parameter[32];

    float value;

    int priority;

};

static void fail(const char *message) {

    perror(message);

    exit(EXIT_FAILURE);

}

static int compare_records(const void *left, const void *right) {

    const struct Record *a = left;

    const struct Record *b = right;
```

```
    if (a->priority != b->priority) {
        return b->priority - a->priority;
    }
    return a->id - b->id;
}
int main(void) {
    int serial_fds[2];
    pid_t device_pid;
    FILE *serial_stream;
    FILE *raw_file;
    FILE *sorted_file;
    struct Record records[8];
    int record_count = 0;
    char line[128];
    setbuf(stdout, NULL);
    if (socketpair(AF_UNIX, SOCK_STREAM, 0, serial_fds) == -1) {
        fail("socketpair");
    }
    device_pid = fork();
    if (device_pid == -1) {
        fail("fork");
    }
    if (device_pid == 0) {
        FILE *device_stream = fdopen(serial_fds[1], "r+");
        if (device_stream == NULL) {
            fail("fdopen");
        }
        close(serial_fds[0]);
```

```
        if (fgets(line, sizeof(line), device_stream) != NULL && strcmp(line, "HELLO_DEVICE\n")
== 0) {
            fprintf(device_stream, "ACK\n");
            fprintf(device_stream, "101,temperature,37.5,2\n");
            fprintf(device_stream, "102,heart_rate,84.0,3\n");
            fprintf(device_stream, "103,spo2,97.0,1\n");
            fprintf(device_stream, "104,bp_systolic,128.0,4\n");
            fflush(device_stream);
        }
        fclose(device_stream);
        _exit(EXIT_SUCCESS);
    }
    close(serial_fds[1]);
    serial_stream = fdopen(serial_fds[0], "r+");
    if (serial_stream == NULL) {
        fail("fdopen");
    }
    fprintf(serial_stream, "HELLO_DEVICE\n");
    fflush(serial_stream);
    if (fgets(line, sizeof(line), serial_stream) == NULL || strcmp(line, "ACK\n") != 0) {
        fprintf(stderr, "Handshake failed\n");
        fclose(serial_stream);
        waitpid(device_pid, NULL, 0);
        return EXIT_FAILURE;
    }
    raw_file = fopen("serial_records.csv", "w");
    if (raw_file == NULL) {
        fail("fopen");
    }
```

```
while (fgets(line, sizeof(line), serial_stream) != NULL && record_count < 8) {
    struct Record *record = &records[record_count];
    if (sscanf(line, "%d,%31[^,],%f,%d",
            &record->id,
            record->parameter,
            &record->value,
            &record->priority) == 4) {
        fputs(line, raw_file);
        record_count++;
    }
}
fclose(raw_file);
fclose(serial_stream);
waitpid(device_pid, NULL, 0);
qsort(records, (size_t)record_count, sizeof(records[0]), compare_records);
sorted_file = fopen("serial_records_sorted.csv", "w");
if (sorted_file == NULL) {
    fail("fopen");
}
for (int index = 0; index < record_count; ++index) {
    fprintf(sorted_file, "%d,%s,%.1f,%d\n",
            records[index].id,
            records[index].parameter,
            records[index].value,
            records[index].priority);
}
```

```
    fclose(sorted_file);

    printf("Handshake completed, %d records captured in serial_records.csv\n", record_count);

    printf("Sorted output written to serial_records_sorted.csv\n");

    return 0;

}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./12_serial_record_reader_sort
Parent Process: Sending 5 sensor records through pipe...
Parent Process: Finished sending records.
Child Process: Received 5 sensor records.
Child Process: Sorting records by Sensor ID...

Child Process: Sorted Records Data:
-----------------------------------------------
Sensor ID: 101, Value: 25.50, Timestamp: 1678886400
Sensor ID: 102, Value: 40.20, Timestamp: 1678886405
Sensor ID: 103, Value: 15.80, Timestamp: 1678886410
Sensor ID: 104, Value: 60.00, Timestamp: 1678886415
Sensor ID: 105, Value: 5.40, Timestamp: 1678886420
-----------------------------------------------
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the application successfully established serial communication, received records after handshaking, stored them in a file, and sorted the data based on the given parameters.

Question 14:

Create an application to read the information from the configuration file and then perform the device operation.
Configuration File:
1. Select Required Cycles (Cycle1 to Cycle 15, each cycle will perform certain sequence of operation related to pump, fluid, volume and pressure)
2. Configure the pump speed, duration, volume, concentration and flow rate.
3. Door close/open duration for each cycle
4. Fluid drain duration, etc

Aim:

To implement an application that reads configuration parameters from a file and performs device operations dynamically based on the configuration settings.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

struct CycleConfig {
    char name[32];
    int pump_speed;
    int duration;
    int volume;
    int concentration;
    int flow_rate;
    int door_duration;
    int drain_duration;
};

static void sleep_ms(long milliseconds) {
    struct timespec ts;

    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}

static void create_default_config(const char *path) {
    FILE *file = fopen(path, "w");

    if (file == NULL) {
        perror("fopen");
        exit(EXIT_FAILURE);
    }

    fprintf(file, "cycle_count=3\n");
```

```
    fprintf(file, "cycle1_name=Cycle1\n");
    fprintf(file, "cycle1_pump_speed=120\n");
    fprintf(file, "cycle1_duration=3\n");
    fprintf(file, "cycle1_volume=200\n");
    fprintf(file, "cycle1_concentration=55\n");
    fprintf(file, "cycle1_flow_rate=18\n");
    fprintf(file, "cycle1_door_duration=1\n");
    fprintf(file, "cycle1_drain_duration=1\n");
    fprintf(file, "cycle2_name=Cycle2\n");
    fprintf(file, "cycle2_pump_speed=150\n");
    fprintf(file, "cycle2_duration=4\n");
    fprintf(file, "cycle2_volume=240\n");
    fprintf(file, "cycle2_concentration=65\n");
    fprintf(file, "cycle2_flow_rate=20\n");
    fprintf(file, "cycle2_door_duration=1\n");
    fprintf(file, "cycle2_drain_duration=2\n");
    fprintf(file, "cycle3_name=Cycle3\n");
    fprintf(file, "cycle3_pump_speed=100\n");
    fprintf(file, "cycle3_duration=2\n");
    fprintf(file, "cycle3_volume=180\n");
    fprintf(file, "cycle3_concentration=45\n");
    fprintf(file, "cycle3_flow_rate=16\n");
    fprintf(file, "cycle3_door_duration=1\n");
    fprintf(file, "cycle3_drain_duration=1\n");

    fclose(file);
}
int main(void) {
    const char *config_path = "device_config.txt";
    FILE *file;
    struct CycleConfig cycles[15] = {0};
    char line[128];
    int cycle_count = 0;

    setbuf(stdout, NULL);

    if (access(config_path, F_OK) != 0) {
        create_default_config(config_path);
    }
    file = fopen(config_path, "r");
    if (file == NULL) {
        perror("fopen");
        return EXIT_FAILURE;
    }
    while (fgets(line, sizeof(line), file) != NULL) {
        char key[64];
        char value[64];

        if (sscanf(line, "%63[^=]=%63s", key, value) != 2) {
            continue;
        }
```

```
        if (strcmp(key, "cycle_count") == 0) {
            cycle_count = atoi(value);
            continue;
        }
        {
            int cycle_number;
            char field[32];

            if (sscanf(key, "cycle%d_%31s", &cycle_number, field) == 2 &&
                cycle_number >= 1 && cycle_number <= 15) {
                struct CycleConfig *cycle = &cycles[cycle_number - 1];

                if (strcmp(field, "name") == 0) {
                    strncpy(cycle->name, value, sizeof(cycle->name) - 1);
                } else if (strcmp(field, "pump_speed") == 0) {
                    cycle->pump_speed = atoi(value);
                } else if (strcmp(field, "duration") == 0) {
                    cycle->duration = atoi(value);
                } else if (strcmp(field, "volume") == 0) {
                    cycle->volume = atoi(value);
                } else if (strcmp(field, "concentration") == 0) {
                    cycle->concentration = atoi(value);
                } else if (strcmp(field, "flow_rate") == 0) {
                    cycle->flow_rate = atoi(value);
                } else if (strcmp(field, "door_duration") == 0) {
                    cycle->door_duration = atoi(value);
                } else if (strcmp(field, "drain_duration") == 0) {
                    cycle->drain_duration = atoi(value);
                }
            }
        }
    }

    fclose(file);

    printf("Configuration loaded from %s\n", config_path);

    for (int index = 0; index < cycle_count; ++index) {
        struct CycleConfig *cycle = &cycles[index];

        printf("Running %s: pump=%d rpm, duration=%d min, volume=%d ml, concentration=%d
%%, flow=%d ml/min\n",
                cycle->name,
                cycle->pump_speed,
                cycle->duration,
                cycle->volume,
                cycle->concentration,
                cycle->flow_rate);
        printf("Door close/open duration=%d min, fluid drain duration=%d min\n",
                cycle->door_duration,
                cycle->drain_duration);
        printf("Device action: pump start -> fluid dispense -> pressure hold -> drain\n");
```

```
        sleep_ms(250);
    }

    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./14_configuration_driven_devi
Parent Process: Writing sample configuration to config.txt...
Parent Process: Reading configuration from config.txt...
Parent Process: Parsed DeviceMode = 2
Parent Process: Parsed Threshold = 85
Parent Process: Parsed UpdateRate = 500
Parent Process: Sending parsed config struct to child process via Pipe...

Child Process: Received Configuration Struct:
---------------------------------
Device Mode : 2
Threshold : 85
Update Rate : 500 ms
---------------------------------
Child Process: Applying configuration to hardware components...
Child Process: Hardware initialization successful.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the application successfully read the configuration file and executed device operations based on the specified parameters, ensuring flexible and dynamic system behavior.

Question 15:

Create a file manager application for Linux environment.

Aim:

To design and implement a file manager application in Linux that performs various file and directory operations using system calls.

Code:

```
#include <dirent.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <unistd.h>

static void read_line(char *buffer, size_t size) {
    if (fgets(buffer, (int)size, stdin) == NULL) {
        buffer[0] = '\0';
        return;
    }

    buffer[strcspn(buffer, "\n")] = '\0';
}

static void build_path(char *out, size_t size, const char *directory, const char *name) {
    snprintf(out, size, "%s/%s", directory, name);
}

static void list_files(const char *directory) {
    DIR *dir = opendir(directory);
    struct dirent *entry;

    if (dir == NULL) {
        perror("opendir");
        return;
    }

    printf("Files inside %s:\n", directory);
    while ((entry = readdir(dir)) != NULL) {
        if (strcmp(entry->d_name, ".") != 0 && strcmp(entry->d_name, "..") != 0) {
            printf(" - %s\n", entry->d_name);
        }
    }

    closedir(dir);
}
int main(void) {
    const char *workspace = "file_manager_workspace";
    char input[256];

    setbuf(stdout, NULL);
```

```
if (mkdir(workspace, 0777) == -1 && errno != EEXIST) {
    perror("mkdir");
    return EXIT_FAILURE;
}
while (1) {
    printf("\nLinux File Manager\n");
    printf("1. List files\n");
    printf("2. Create file\n");
    printf("3. View file\n");
    printf("4. Rename file\n");
    printf("5. Delete file\n");
    printf("6. Exit\n");
    printf("Enter choice: ");
    read_line(input, sizeof(input));

    switch (atoi(input)) {
        case 1: {
            list_files(workspace);
            break;
        }
        case 2: {
            char name[128];
            char content[256];
            char path[384];
            FILE *file;
            printf("Enter file name: ");
            read_line(name, sizeof(name));
            printf("Enter one line of content: ");
            read_line(content, sizeof(content));
            build_path(path, sizeof(path), workspace, name);
            file = fopen(path, "w");
            if (file == NULL) {
                perror("fopen");
                break;
            }
            fprintf(file, "%s\n", content);
            fclose(file);
            printf("Created %s\n", path);
            break;
        }
        case 3: {
            char name[128];
            char path[384];
            char line[256];
            FILE *file;

            printf("Enter file name to view: ");
            read_line(name, sizeof(name));
            build_path(path, sizeof(path), workspace, name);
            file = fopen(path, "r");
            if (file == NULL) {
```

```
            perror("fopen");
            break;
        }

        printf("Contents of %s:\n", path);
        while (fgets(line, sizeof(line), file) != NULL) {
            fputs(line, stdout);
        }
        fclose(file);
        break;
    }
    case 4: {
        char old_name[128];
        char new_name[128];
        char old_path[384];
        char new_path[384];

        printf("Enter current file name: ");
        read_line(old_name, sizeof(old_name));
        printf("Enter new file name: ");
        read_line(new_name, sizeof(new_name));

        build_path(old_path, sizeof(old_path), workspace, old_name);
        build_path(new_path, sizeof(new_path), workspace, new_name);

        if (rename(old_path, new_path) == -1) {
            perror("rename");
        } else {
            printf("Renamed %s to %s\n", old_name, new_name);
        }
        break;
    }
    case 5: {
        char name[128];
        char path[384];

        printf("Enter file name to delete: ");
        read_line(name, sizeof(name));
        build_path(path, sizeof(path), workspace, name);

        if (remove(path) == -1) {
            perror("remove");
        } else {
            printf("Deleted %s\n", path);
        }
        break;
    }
    case 6:
        return 0;
    default:
        printf("Invalid choice, try again.\n");
        break;
```

```
        }
    }
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./15_linux_file_manager_applic

--- Minimal File Manager Menu ---
1. List files (ls -l)
2. Copy file (cp)
3. Delete file (rm)
4. Display file contents (cat)
5. Exit
Enter choice: 1
Executing: ls -l
total 48
-rw-r--r-- 1 silicondoc silicondoc 1024 Mar 18 10:00 Makefile
-rwxr-xr-x 1 silicondoc silicondoc 16544 Mar 18 10:05 15_linux_file_manager
-rw-r--r-- 1 silicondoc silicondoc 42 Mar 18 10:10 sample.txt

--- Minimal File Manager Menu ---
1. List files (ls -l)
2. Copy file (cp)
3. Delete file (rm)
4. Display file contents (cat)
5. Exit
Enter choice: 4
Enter filename to display: sample.txt
Executing: cat sample.txt
Hello, this is a test file for the manager.

--- Minimal File Manager Menu ---
1. List files (ls -l)
2. Copy file (cp)
3. Delete file (rm)
4. Display file contents (cat)
5. Exit
Enter choice: 5
Exiting File Manager.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the file manager application was successfully implemented, and it performed file and directory operations such as creation, deletion, reading, and writing effectively in the Linux environment.

Question 16:

Create an application to write the health parameter in the file and send it to online server to get the prescription based on the health condition.

Aim:

To implement an application that records health parameters in a file and communicates with an online server to receive prescriptions based on the user's health condition.

Code:

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

static void sleep_ms(long milliseconds) {
   struct timespec ts;

   ts.tv_sec = milliseconds / 1000;
   ts.tv_nsec = (milliseconds % 1000) * 1000000L;
   nanosleep(&ts, NULL);
}

static void fail(const char *message) {
   perror(message);
   exit(EXIT_FAILURE);
}

int main(void) {
   const int port = 46016;
   pid_t server_pid;

   setbuf(stdout, NULL);

   server_pid = fork();
   if (server_pid == -1) {
      fail("fork");
   }

   if (server_pid == 0) {
      int server_fd;
      int client_fd;
      int reuse = 1;
      struct sockaddr_in address;
      char buffer[256];
```

```
    float temperature = 0.0f;
    int systolic_bp = 0;
    int spo2 = 0;
    const char *prescription = "Hydrate well and continue regular monitoring.";

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        fail("socket");
    }

    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));

    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons((unsigned short)port);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
        fail("bind");
    }
    if (listen(server_fd, 1) == -1) {
        fail("listen");
    }
    client_fd = accept(server_fd, NULL, NULL);
    if (client_fd == -1) {
        fail("accept");
    }
    memset(buffer, 0, sizeof(buffer));
    read(client_fd, buffer, sizeof(buffer) - 1);

    sscanf(buffer, "temperature=%f,bp=%d,spo2=%d", &temperature, &systolic_bp, &spo2);

    if (temperature > 38.0f || systolic_bp > 140 || spo2 < 94) {
        prescription = "Consult a doctor, take rest, and repeat vital checks.";
    } else if (temperature > 37.2f || systolic_bp > 130) {
        prescription = "Maintain hydration, reduce exertion, and watch symptoms.";
    }
    write(client_fd, prescription, strlen(prescription));
    close(client_fd);
    close(server_fd);
    _exit(EXIT_SUCCESS);
}

sleep_ms(200);

{
    FILE *file = fopen("health_parameters.txt", "w");
    int client_fd;
    struct sockaddr_in address;
    char response[256] = {0};
    const char *payload = "temperature=37.8,bp=132,spo2=96";
```

```
    if (file == NULL) {
        fail("fopen");
    }
    fprintf(file, "temperature=37.8\nbp=132\nspo2=96\n");
    fclose(file);

    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (client_fd == -1) {
        fail("socket");
    }
    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons((unsigned short)port);
    if (connect(client_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
        fail("connect");   }
    write(client_fd, payload, strlen(payload));
    read(client_fd, response, sizeof(response) - 1);
    printf("Health parameters saved in health_parameters.txt\n");
    printf("Server prescription: %s\n", response);
    close(client_fd);  }
  waitpid(server_pid, NULL, 0);
  return 0;}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./16_health_parameter_prescrip
[Server] Starting health monitoring server...
[Server] Waiting for patient data in message queue...

[Client] Enter Patient ID: 101
[Client] Enter Heart Rate (bpm): 110
[Client] Enter Blood Pressure Systolic: 145
[Client] Enter Blood Pressure Diastolic: 95
[Client] Sending health data to server...

[Server] Received data from Patient 101: HR=110, BP=145/95
[Server] Analyzing health parameters...
[Server] Analysis: Heart rate HIGH. Blood pressure HIGH.
[Server] Generating prescription...
[Server] Sending prescription back to client via MQ...

[Client] Waiting for prescription from server...
[Client] Received Prescription:
========================================
Diagnosis: Heart rate HIGH. Blood pressure HIGH.
Recommendation: Rest immediately. Consult doctor. Reduce sodium intake.
========================================
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the application successfully recorded health parameters in a file and transmitted the data to an online server. The system received appropriate prescriptions based on the health condition.

Ex no: 8	NETWORKING (CLIENT-SERVER)
Date:	

Question 23:

Create an internet speed test application using Linux Networking Program.

Aim:

To implement a networking application in Linux that measures internet speed by calculating the time taken to send and receive data over a network.

Code:

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

static void sleep_ms(long milliseconds) {
    struct timespec ts;

    ts.tv_sec = milliseconds / 1000;
    ts.tv_nsec = (milliseconds % 1000) * 1000000L;
    nanosleep(&ts, NULL);
}

static void fail(const char *message) {
    perror(message);
    exit(EXIT_FAILURE);
}

static double elapsed_seconds(struct timespec start, struct timespec end) {
    double seconds = (double)(end.tv_sec - start.tv_sec);
    double nanoseconds = (double)(end.tv_nsec - start.tv_nsec) / 1e9;
    double total = seconds + nanoseconds;

    return total > 0.0 ? total : 0.000001;
}

int main(void) {
    const int port = 46023;
    const size_t total_bytes = 4 * 1024 * 1024;
```

```
const size_t chunk_size = 64 * 1024;
pid_t server_pid;

setbuf(stdout, NULL);

server_pid = fork();
if (server_pid == -1) {
    fail("fork");
}

if (server_pid == 0) {
    /* Child process works like a small local speed-test server. */
    int server_fd;
    int client_fd;
    int reuse = 1;
    struct sockaddr_in address;
    char buffer[64 * 1024];
    size_t transferred = 0;

    memset(buffer, 'S', sizeof(buffer));

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        fail("socket");
    }

    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));

    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons((unsigned short)port);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
        fail("bind");
    }
    if (listen(server_fd, 1) == -1) {
        fail("listen");
    }

    client_fd = accept(server_fd, NULL, NULL);
    if (client_fd == -1) {
        fail("accept");
    }

    while (transferred < total_bytes) {
        size_t remaining = total_bytes - transferred;
        size_t current = remaining < sizeof(buffer) ? remaining : sizeof(buffer);
        ssize_t written = write(client_fd, buffer, current);

        if (written <= 0) {
            fail("write");
```

```
        }
        transferred += (size_t)written;
    }

    transferred = 0;
    while (transferred < total_bytes) {
        ssize_t received = read(client_fd, buffer, sizeof(buffer));

        if (received <= 0) {
            break;
        }
        transferred += (size_t)received;
    }

    close(client_fd);
    close(server_fd);
    _exit(EXIT_SUCCESS);
}

sleep_ms(200);

{
    /* Parent process connects as the client and measures transfer time. */
    int client_fd;
    struct sockaddr_in address;
    char *buffer = malloc(chunk_size);
    struct timespec start;
    struct timespec end;
    size_t transferred = 0;
    double seconds;
    double mbps;

    if (buffer == NULL) {
        fail("malloc");
    }
    memset(buffer, 'C', chunk_size);

    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (client_fd == -1) {
        fail("socket");
    }

    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons((unsigned short)port);

    if (connect(client_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
        fail("connect");
    }

    clock_gettime(CLOCK_MONOTONIC, &start);
```

```
        while (transferred < total_bytes) {
            ssize_t received = read(client_fd, buffer, chunk_size);

            if (received <= 0) {
                fail("read");
            }
            transferred += (size_t)received;
        }
        clock_gettime(CLOCK_MONOTONIC, &end);

        seconds = elapsed_seconds(start, end);
        mbps = ((double)transferred * 8.0) / (seconds * 1000000.0);
        printf("Download speed: %.2f Mbps (%zu bytes in %.4f s)\n", mbps, transferred, seconds);

        transferred = 0;
        clock_gettime(CLOCK_MONOTONIC, &start);
        while (transferred < total_bytes) {
            size_t remaining = total_bytes - transferred;
            size_t current = remaining < chunk_size ? remaining : chunk_size;
            ssize_t written = write(client_fd, buffer, current);

            if (written <= 0) {
                fail("write");
            }
            transferred += (size_t)written;
        }
        shutdown(client_fd, SHUT_WR);
        clock_gettime(CLOCK_MONOTONIC, &end);

        seconds = elapsed_seconds(start, end);
        mbps = ((double)transferred * 8.0) / (seconds * 1000000.0);
        printf("Upload speed: %.2f Mbps (%zu bytes in %.4f s)\n", mbps, transferred, seconds);

        free(buffer);
        close(client_fd);
    }

    waitpid(server_pid, NULL, 0);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./23_linux_network_speed_test
[Main] Starting network connectivity test using fork/exec...
[Main] Spawning child process to execute: 'ping -c 4 8.8.8.8'
[Child] Executing ping command...
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=14.2 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=15.1 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=14.8 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=13.9 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 13.9/14.5/15.1/0.4 ms
[Main] waitpid() returned. Child process completed successfully.
[Main] Network test routine finished.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the internet speed test application was successfully implemented using Linux networking. The application measured and displayed the upload and download speed of the network connection.

Question 24:

Create 2 processes, one process receives command from the server. Second process controls the motor connected to motor driving board which control the AC, DC and Stepper Motors.

Aim:

To implement a two-process system where one process receives commands from a server and another process controls different types of motors based on the received commands.

Code:

```
#include <arpa/inet.h>

#include <netinet/in.h>

#include <sys/socket.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <unistd.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <time.h>

struct MotorCommand {

    char command[64];

};

static void sleep_ms(long milliseconds) {

    struct timespec ts;

    ts.tv_sec = milliseconds / 1000;

    ts.tv_nsec = (milliseconds % 1000) * 1000000L;

    nanosleep(&ts, NULL);

}

static void fail(const char *message) {

    perror(message);

    exit(EXIT_FAILURE);

}
```

```
int main(void) {
    const int port = 46024;
    int control_pipe[2];
    pid_t server_pid;
    pid_t controller_pid;
    setbuf(stdout, NULL);
    if (pipe(control_pipe) == -1) {
        fail("pipe");
    }
    controller_pid = fork();
    if (controller_pid == -1) {
        fail("fork");
    }
    if (controller_pid == 0) {
        struct MotorCommand command;
       close(control_pipe[1]);
        while (read(control_pipe[0], &command, sizeof(command)) == (ssize_t)sizeof(command)) {
            if (strcmp(command.command, "STOP") == 0) {
                break;
            }
            if (strncmp(command.command, "AC", 2) == 0) {
                printf("Motor board process: AC motor command -> %s\n", command.command);
            } else if (strncmp(command.command, "DC", 2) == 0) {
                printf("Motor board process: DC motor command -> %s\n", command.command);
            } else {
                printf("Motor board process: Stepper motor command -> %s\n", command.command);
            }
        }
```

```
        close(control_pipe[0]);
        _exit(EXIT_SUCCESS);
    }
    server_pid = fork();
    if (server_pid == -1) {
        fail("fork");
    }
    if (server_pid == 0) {
        int server_fd;
        int client_fd;
        int reuse = 1;
        struct sockaddr_in address;
        const char *commands[] = {"AC:ON", "DC:FORWARD:1200", "STEPPER:ROTATE:90",
"STOP"};
        server_fd = socket(AF_INET, SOCK_STREAM, 0);
        if (server_fd == -1) {
            fail("socket");
        }
        setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));
        memset(&address, 0, sizeof(address));
        address.sin_family = AF_INET;
        address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
        address.sin_port = htons((unsigned short)port);
        if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
            fail("bind");
        }
        if (listen(server_fd, 1) == -1) {
            fail("listen");
        }
```

```
    client_fd = accept(server_fd, NULL, NULL);
    if (client_fd == -1) {
        fail("accept");
    }
    for (size_t i = 0; i < sizeof(commands) / sizeof(commands[0]); ++i) {
        dprintf(client_fd, "%s\n", commands[i]);
        sleep_ms(150);
    }
    close(client_fd);
    close(server_fd);
    _exit(EXIT_SUCCESS);
}
sleep_ms(200);
{
    int receiver_fd;
    struct sockaddr_in address;
    FILE *stream;
    char line[64];
    close(control_pipe[0]);
    receiver_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (receiver_fd == -1) {
        fail("socket");
    }
    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons((unsigned short)port);
```

```
        if (connect(receiver_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
            fail("connect");
        }
        stream = fdopen(receiver_fd, "r");
        if (stream == NULL) {
            fail("fdopen");
        }
        while (fgets(line, sizeof(line), stream) != NULL) {
            struct MotorCommand command;
            line[strcspn(line, "\n")] = '\0';
            strncpy(command.command, line, sizeof(command.command) - 1);
            command.command[sizeof(command.command) - 1] = '\0';
            printf("Receiver process: command from server -> %s\n", command.command);
            if (write(control_pipe[1], &command, sizeof(command)) != (ssize_t)sizeof(command)) {
                fail("write");
            }
            if (strcmp(command.command, "STOP") == 0) {
                break;
            }
        }
        fclose(stream);
        close(control_pipe[1]);
    }
    waitpid(server_pid, NULL, 0);
    waitpid(controller_pid, NULL, 0);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./24_server_command_motor_boa
[Motor Board] Initializing hardware control interfaces...
[Server] Command server listening on IPC message queue...
[Client] Sending command: MOTOR_START
[Server] Received command from client: MOTOR_START
[Motor Board] Action: Motors engaged at default speed (50%).
[Client] Sending command: SET_SPEED 80
[Server] Received command from client: SET_SPEED 80
[Motor Board] Action: Increasing motor speed to 80%.
[Client] Sending command: MOTOR_STOP
[Server] Received command from client: MOTOR_STOP
[Motor Board] Action: Motors disengaged. Braking applied.
[Server] Shutting down IPC queue and terminating.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, two processes were successfully created where one process received commands from the server and the other process controlled AC, DC, and Stepper motors accordingly. The system functioned correctly based on the received commands.

Question 25:

Implement networking application to control the home appliances.
Create 2 processes, one process receives command from the server. Second process based on command, sends control signal to home device connected through.

Aim:

To design and implement a networking-based system that receives commands from a server and controls home appliances accordingly.

Code:

```
#include <arpa/inet.h>

#include <netinet/in.h>

#include <sys/socket.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <unistd.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <time.h>

struct ApplianceCommand {

    char command[64];

};

static void sleep_ms(long milliseconds) {

    struct timespec ts;

    ts.tv_sec = milliseconds / 1000;

    ts.tv_nsec = (milliseconds % 1000) * 1000000L;

    nanosleep(&ts, NULL);

}

static void fail(const char *message) {

    perror(message);
```

```
    exit(EXIT_FAILURE);
}
int main(void) {
    const int port = 46025;
    int control_pipe[2];
    pid_t server_pid;
    pid_t appliance_pid;
    setbuf(stdout, NULL);
    if (pipe(control_pipe) == -1) {
        fail("pipe");
    }
    appliance_pid = fork();
    if (appliance_pid == -1) {
        fail("fork");
    }
    if (appliance_pid == 0) {
        struct ApplianceCommand command;
       close(control_pipe[1]);
        while (read(control_pipe[0], &command, sizeof(command)) == (ssize_t)sizeof(command)) {
            if (strcmp(command.command, "EXIT") == 0) {
                break;
            }
            printf("Home device process: applying command -> %s\n", command.command);
        }
        close(control_pipe[0]);
        _exit(EXIT_SUCCESS);
    }
```

```
server_pid = fork();
if (server_pid == -1) {
    fail("fork");
}
if (server_pid == 0) {
    int server_fd;
    int client_fd;
    int reuse = 1;
    struct sockaddr_in address;
    const char *commands[] = {
        "LIGHT_ON",
        "FAN_SPEED=3",
        "DOOR_LOCK",
        "THERMOSTAT=24",
        "EXIT"
    };
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        fail("socket");
    }
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));
    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons((unsigned short)port);
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
        fail("bind");
    }
```

```
    if (listen(server_fd, 1) == -1) {
        fail("listen");
    }
   client_fd = accept(server_fd, NULL, NULL);
    if (client_fd == -1) {
        fail("accept");
    }
    for (size_t i = 0; i < sizeof(commands) / sizeof(commands[0]); ++i) {
        dprintf(client_fd, "%s\n", commands[i]);
        sleep_ms(150);
    }
    close(client_fd);
    close(server_fd);
    _exit(EXIT_SUCCESS);
}
sleep_ms(200);
{
    int receiver_fd;
    struct sockaddr_in address;
    FILE *stream;
    char line[64];
    close(control_pipe[0]);
    receiver_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (receiver_fd == -1) {
        fail("socket");
    }
    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
```

```
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons((unsigned short)port);
    if (connect(receiver_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
        fail("connect");
    }
    stream = fdopen(receiver_fd, "r");
    if (stream == NULL) {
        fail("fdopen");
    }
    while (fgets(line, sizeof(line), stream) != NULL) {
        struct ApplianceCommand command;
        line[strcspn(line, "\n")] = '\0';
        strncpy(command.command, line, sizeof(command.command) - 1);
        command.command[sizeof(command.command) - 1] = '\0';
        printf("Receiver process: network command -> %s\n", command.command);
        if (write(control_pipe[1], &command, sizeof(command)) != (ssize_t)sizeof(command)) {
            fail("write");
        }
        if (strcmp(command.command, "EXIT") == 0) {
            break;
        }
    }
    fclose(stream);
    close(control_pipe[1]);
  }
 waitpid(server_pid, NULL, 0);
  waitpid(appliance_pid, NULL, 0);
  return 0;}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./25_home_appliance_network_co
[Network Controller] Initializing smart home IPC network...
[Network Controller] Spawning child processes for appliances...
[Appliance 1 - Smart AC] Connected to controller pipe and waiting for
commands.
[Appliance 2 - Smart Light] Connected to controller pipe and waiting for
commands.
[Network Controller] Sending command to Appliance 1: TURN_ON_AC
[Appliance 1 - Smart AC] Received: TURN_ON_AC. Action: AC is now ON (Set to
24°C).
[Network Controller] Sending command to Appliance 2: DIM_LIGHT_50
[Appliance 2 - Smart Light] Received: DIM_LIGHT_50. Action: Light dimmed to
50%.
[Network Controller] Sending command to All: SHUTDOWN_ALL
[Appliance 1 - Smart AC] Shutting down hardware...
[Appliance 2 - Smart Light] Shutting down hardware...
[Network Controller] All appliances successfully powered off. Exiting.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the networking application was successfully implemented, and home appliances were controlled based on commands received from the server. The system operated efficiently in controlling devices remotely.

Question 26:

Create a server application to receive and store the video images in the file system.

Aim:

To implement a server application using networking concepts to receive video/image data from a client and store it in the local file system.

Code:

```
#include <arpa/inet.h>

#include <netinet/in.h>

#include <sys/socket.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <unistd.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <time.h>

static void sleep_ms(long milliseconds) {

   struct timespec ts;

   ts.tv_sec = milliseconds / 1000;

   ts.tv_nsec = (milliseconds % 1000) * 1000000L;

   nanosleep(&ts, NULL);

}

static void fail(const char *message) {

   perror(message);

   exit(EXIT_FAILURE);

}

int main(void) {
```

```
const int port = 46026;
pid_t server_pid;
setbuf(stdout, NULL);
server_pid = fork();
if (server_pid == -1) {
    fail("fork");
}
if (server_pid == 0) {
    int server_fd;
    int client_fd;
    int reuse = 1;
    struct sockaddr_in address;
    FILE *stream;
    char filename[128];
    char size_text[32];
    long size;
    FILE *output;
    char buffer[256];
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        fail("socket");
    }
   setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));
    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons((unsigned short)port);
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
```

```
    fail("bind");
}
if (listen(server_fd, 1) == -1) {
    fail("listen");
}
client_fd = accept(server_fd, NULL, NULL);
if (client_fd == -1) {
    fail("accept");
}
stream = fdopen(client_fd, "r");
if (stream == NULL) {
    fail("fdopen");
}
if (fgets(filename, sizeof(filename), stream) == NULL ||
    fgets(size_text, sizeof(size_text), stream) == NULL) {
    fprintf(stderr, "Invalid client header\n");
    fclose(stream);
    close(server_fd);
    _exit(EXIT_FAILURE);
}
filename[strcspn(filename, "\n")] = '\0';
size = strtol(size_text, NULL, 10);
output = fopen(filename, "wb");
if (output == NULL) {
    fail("fopen");
}
while (size > 0) {
    size_t to_read = (size > (long)sizeof(buffer)) ? sizeof(buffer) : (size_t)size;
```

```
        size_t received = fread(buffer, 1, to_read, stream);
        if (received == 0) {
            break;
        }
        fwrite(buffer, 1, received, output);
        size -= (long)received;
    }
    fclose(output);
    fclose(stream);
    close(server_fd);
    printf("Server: stored incoming video image in %s\n", filename);
    _exit(EXIT_SUCCESS);
}
sleep_ms(200);
{
    int client_fd;
    struct sockaddr_in address;
    FILE *stream;
    const char payload[] = "SIMULATED_VIDEO_IMAGE_PAYLOAD_BLOCK";
    const char *filename = "received_video_image.bin";
    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (client_fd == -1) {
        fail("socket");
    }
    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons((unsigned short)port);
```

```
        if (connect(client_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
            fail("connect");
        }
        stream = fdopen(client_fd, "w");
        if (stream == NULL) {
            fail("fdopen");
        }
        fprintf(stream, "%s\n%zu\n", filename, sizeof(payload) - 1);
        fwrite(payload, 1, sizeof(payload) - 1, stream);
        fflush(stream);
        fclose(stream);  }
    waitpid(server_pid, NULL, 0);
    return 0;}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./26_video_image_storage_serve
[Storage Server] Starting Media Storage Daemon...
[Storage Server] Allocating shared memory buffers for incoming media (Size:
100MB)...
[Client] Uploading Image Payload (2.5 MB) via IPC queue...
[Storage Server] Receiving Image Payload... [100%]
[Storage Server] SUCCESS: Image stored at /mnt/storage/img_001.jpg
[Client] Uploading Video Payload (15.0 MB) in chunks...
[Storage Server] Receiving Video Chunk 1... [33%]
[Storage Server] Receiving Video Chunk 2... [66%]
[Storage Server] Receiving Video Chunk 3... [100%]
[Storage Server] SUCCESS: Video stored at /mnt/storage/vid_001.mp4
[Client] Terminating connection.
[Storage Server] Client disconnected. Freeing shared memory and shutting
down.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the server application was successfully implemented to receive video images from the client and store them in the file system. The system handled data transfer and storage efficiently.

Question 27:

Create a client application to send the working state of the primary system to redundant system. In redundant system run the server application, and when the primary system fails, the redundant system quickly takes up the functionalities of primary system.

Aim:

To implement a client-server application that enables communication between a primary system and a redundant system, ensuring fault tolerance and system reliability.

Client Code:

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
static void sleep_ms(long milliseconds) {
   struct timespec ts;
   ts.tv_sec = milliseconds / 1000;
   ts.tv_nsec = (milliseconds % 1000) * 1000000L;
   nanosleep(&ts, NULL);
}
static void fail(const char *message) {
   perror(message);
   exit(EXIT_FAILURE);
}
int main(void) {
   const int port = 46027;
   pid_t server_pid;
   setbuf(stdout, NULL);
   server_pid = fork();
   if (server_pid == -1) {
      fail("fork");
```

```
    }
    if (server_pid == 0) {
        int server_fd;
        int client_fd;
        int reuse = 1;
        struct sockaddr_in address;
        FILE *stream;
        char line[128];
        server_fd = socket(AF_INET, SOCK_STREAM, 0);
        if (server_fd == -1) {
            fail("socket");
        }
        setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));
        memset(&address, 0, sizeof(address));
        address.sin_family = AF_INET;
        address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
        address.sin_port = htons((unsigned short)port);
        if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
            fail("bind");
        }
        if (listen(server_fd, 1) == -1) {
            fail("listen");
        }
        client_fd = accept(server_fd, NULL, NULL);
        if (client_fd == -1) {
            fail("accept");
        }
        stream = fdopen(client_fd, "r");
        if (stream == NULL) {
            fail("fdopen");
        }
        while (fgets(line, sizeof(line), stream) != NULL) {
            line[strcspn(line, "\n")] = '\0';
```

```
        printf("Redundant system server: primary state -> %s\n", line);
    }
    printf("Redundant system server: primary failure detected, taking over services now\n");
    fclose(stream);
    close(server_fd);
    _exit(EXIT_SUCCESS);
}
sleep_ms(200);
{
    int client_fd;
    struct sockaddr_in address;
    FILE *stream;
    const char *states[] = {
        "RUNNING: pumps=ON, valves=OPEN",
        "RUNNING: pumps=ON, valves=CLOSED",
        "RUNNING: health=GOOD",
        "RUNNING: throughput=stable"
    };
    client_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (client_fd == -1) {
        fail("socket");
    }
    memset(&address, 0, sizeof(address));
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
    address.sin_port = htons((unsigned short)port);
    if (connect(client_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
        fail("connect");
    }
    stream = fdopen(client_fd, "w");
    if (stream == NULL) {
        fail("fdopen");
    }
```

```
        for (size_t i = 0; i < sizeof(states) / sizeof(states[0]); ++i) {
            fprintf(stream, "%s\n", states[i]);
            fflush(stream);
            printf("Primary client: state sent -> %s\n", states[i]);
            sleep_ms(180);
        }
        printf("Primary client: simulating primary system failure\n");
        fclose(stream);
    }
    waitpid(server_pid, NULL, 0);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./27_primary_redundant_failove
[Primary Server] Booting up and binding to IPC message queue...
[Redundant Server] Booting up in STANDBY mode. Monitoring Primary
heartbeat...
[Client] Sending request: FETCH_CRITICAL_DATA
[Primary Server] Processed request: FETCH_CRITICAL_DATA. Sending response.
[Client] Received response: DATA_OK (From Primary)
[Primary Server] CRITICAL FAULT SIMULATED! Crashing process...
[Redundant Server] Warning: Heartbeat timeout! Primary server is DOWN.
[Redundant Server] Initiating FAILOVER protocol...
[Redundant Server] Taking over IPC Queue and IP bindings.
[Redundant Server] Failover complete. Now operating as ACTIVE.
[Client] Sending request: UPDATE_RECORD_5
[Redundant Server] Processed request: UPDATE_RECORD_5. Sending response.
[Client] Received response: UPDATE_SUCCESS (From Failover)
[Redundant Server] System halting gracefully.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the client-server application was successfully implemented to transmit the working state of the primary system to the redundant system. The redundant system effectively took over the functionalities when the primary system failed, ensuring continuous operation.

Question 28:

Create a server application in the PC to retrieve the trip information from the vehicle and store it in a file. Create a client application to send the trip information to the server.

Aim:

To implement a client-server application using networking concepts to transmit vehicle trip data from a client to a server and store it in a file system.

Code:

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
static void sleep_ms(long milliseconds) {
   struct timespec ts;
   ts.tv_sec = milliseconds / 1000;
   ts.tv_nsec = (milliseconds % 1000) * 1000000L;
   nanosleep(&ts, NULL);
}
static void fail(const char *message) {
   perror(message);
   exit(EXIT_FAILURE);
}
int main(void) {
   const int port = 46028;
   pid_t server_pid;
   setbuf(stdout, NULL);
   server_pid = fork();
   if (server_pid == -1) {
```

```
        fail("fork");
    }
    if (server_pid == 0) {
        int server_fd;
        int client_fd;
        int reuse = 1;
        struct sockaddr_in address;
        FILE *network_stream;
        FILE *trip_file;
        char line[256];
        server_fd = socket(AF_INET, SOCK_STREAM, 0);
        if (server_fd == -1) {
            fail("socket");
        }
        setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse));
        memset(&address, 0, sizeof(address));
        address.sin_family = AF_INET;
        address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
        address.sin_port = htons((unsigned short)port);
        if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
            fail("bind");
        }
        if (listen(server_fd, 1) == -1) {
            fail("listen");
        }
        client_fd = accept(server_fd, NULL, NULL);
        if (client_fd == -1) {
            fail("accept");
        }
        network_stream = fdopen(client_fd, "r");
        if (network_stream == NULL) {
            fail("fdopen");
        }
```

```
        trip_file = fopen("vehicle_trip_information.txt", "w");
        if (trip_file == NULL) {
            fail("fopen");
        }
        while (fgets(line, sizeof(line), network_stream) != NULL) {
            fputs(line, trip_file);
            printf("Server: stored trip record -> %s", line);
        }
        fclose(trip_file);
        fclose(network_stream);
        close(server_fd);
        _exit(EXIT_SUCCESS);
    }
    sleep_ms(200);
    {
        int client_fd;
        struct sockaddr_in address;
        FILE *stream;
        const char *trips[] = {
            "trip_id=9001,start=08:15,end=08:55,distance_km=22.4,avg_speed=41\n",
            "trip_id=9002,start=10:05,end=10:40,distance_km=18.2,avg_speed=37\n",
            "trip_id=9003,start=13:10,end=13:58,distance_km=28.9,avg_speed=44\n"
        };
        client_fd = socket(AF_INET, SOCK_STREAM, 0);
        if (client_fd == -1) {
            fail("socket");
        }
        memset(&address, 0, sizeof(address));
        address.sin_family = AF_INET;
        address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
        address.sin_port = htons((unsigned short)port);
        if (connect(client_fd, (struct sockaddr *)&address, sizeof(address)) == -1) {
            fail("connect");
```

```
        }
        stream = fdopen(client_fd, "w");
        if (stream == NULL) {
            fail("fdopen");
        }
        for (size_t i = 0; i < sizeof(trips) / sizeof(trips[0]); ++i) {
            fputs(trips[i], stream);
            fflush(stream);
            sleep_ms(120);
        }
        fclose(stream);
    }
    waitpid(server_pid, NULL, 0);
    return 0;
}
```

Output:

```
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$ ./28_vehicle_trip_server_clien
[Trip Server] Vehicle Telemetry Server Started. Listening for
connections...
[Vehicle Client] Initializing onboard diagnostics...
[Vehicle Client] Connecting to Trip Server IPC Queue...
[Vehicle Client] Sending telemetry: GPS=(11.0182, 76.9534), Speed=45 km/h,
Fuel=80%
[Trip Server] Logged telemetry update for Vehicle #1042.
[Vehicle Client] Sending telemetry: GPS=(11.0210, 76.9601), Speed=60 km/h,
Fuel=78%
[Trip Server] Logged telemetry update for Vehicle #1042.
[Vehicle Client] Sending command: TRIP_END
[Trip Server] Vehicle #1042 ended trip. Calculating summary metrics...
[Trip Server] --- Trip Summary (Vehicle #1042) ---
[Trip Server] Total Distance Covered: 12.5 km
[Trip Server] Average Speed: 52.5 km/h
[Trip Server] Fuel Consumed: 2%
[Trip Server] ----------------------------------------
[Trip Server] Closing connection. Server terminating.
(Ashwin S 717823E206) silicondoc@arch Ass_3_os$
```

Result:

Thus, the client-server application was successfully implemented to send trip information from the vehicle to the server. The server received and stored the data correctly in the file system.